

A Multi-Layer Framework for Hallucination Detection and Mitigation in Large Language Models Using Retrieval Grounding and Small Language Model Verification

Md. Mahmudul Hasan

mahmudulhasan@bou.ac.bd

Bangladesh Open University

Israk Kayum Chowdhury

Bangladesh Open University

Md. Eliyas Akondo

Bangladesh Open University

Tanvir Ahammad

Jagannath University

Khandaker Mohammad Mohi Uddin

Southeast University

Research Article

Keywords: Claim-Level Verification, Semantic Grounding, Neuro-Symbolic Reasoning, Trust Calibration, Uncertainty-Aware AI, Retrieval-Augmented Generation (RAG)

Posted Date: June 25th, 2026

DOI: <https://doi.org/10.21203/rs.3.rs-9986071/v1>

License: © ⓘ This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: No competing interests reported.

A Multi-Layer Framework for Hallucination Detection and Mitigation in Large Language Models Using Retrieval Grounding and Small Language Model Verification

Anonymous Author^{1*}

^{1*} Anonymous Department, Anonymous Institution, Anonymous City, Anonymous Country.

Abstract

Large language models can produce fluent answers, but some of their statements may be unsupported, incomplete, or factually wrong. This problem becomes more serious when users used such models and expect reliable information. In this paper, we propose a multi-layer framework that checks generated answers at the claim level instead of judging the whole response at once. The framework first breaks an LLM-generated response into atomic factual claims then retrieves relevant evidence for each claim. After that, it verifies the claim with a small language model, and uses a neuro-symbolic reasoning layer to refine uncertain or incorrect cases. This process identifies specific factual errors, handle weak evidence more cautiously, and revise unsupported claims with explanations. We analyze this framework on HotpotQA and SciFact datasets to test both multi-hop question answering and scientific claim verification. On HotpotQA, the full pipeline achieved a token-level F1 score of 0.146. The score dropped to 0.034 when retrieval was removed which shows how strongly the system depends on evidence grounding. The best F1 score near 0.40 gave from threshold analysis, but the final system was stricter threshold near 0.60 to reduce false supported predictions. On SciFact, the framework achieved 0.511 accuracy and 0.504 macro-F1. The no-reasoner variant performed almost the same as the full framework, suggesting that the SLM verifier carried most of the structured classification work. The ROC-AUC value was about 0.87 which are useful confidence behaviour. Overall, the results show that the retrieval grounding and lightweight verification can improve factual reliability while keeping the system interpretable and computationally practical.

Keywords: Claim-Level Verification, Semantic Grounding, Neuro-Symbolic Reasoning, Trust Calibration, Uncertainty-Aware AI, Retrieval-Augmented Generation (RAG)

1 Introduction

Large Language Models (LLMs) have demonstrated promising results in diverse natural language processing applications. They are now routinely used both in research and in real-world applications. They have also been used in actual systems in recent years. They can write text that frequently reads fluently, makes sense and mimics human writing. This has enabled them to be used in education, healthcare, and more, and decision support. But in these places, the ability to communicate fluently is not sufficient. The information must also be accurate, as it can cause serious misunderstandings if it contains minor inaccuracies. They can affect the decision-making process of users or important decisions.

Yet, LLMs are very bad at giving answers that sound correct but are not. This is frequently referred to as “hallucination”. In reality, a hallucinated answer can include a wrong date, a fictional reference, an incorrect medical statement or a statement that cannot be substantiated from a reliable source. The tricky thing is that these errors can appear in well-worded and confident language. This confidence may be misleading in domains such as health sciences, science writing, education or science policy, where a minor factual error can alter the interpretation and/or behavior of a user.

There has been recent research on LLM hallucination in multiple ways [1], [2]. Some of this research focuses primarily on the detection of hallucination as a problem. The methods attempt to identify unreliable outputs using various approaches like self-consistency analysis, multi-perspective reasoning and exploration of internal model representations [3], [4], [5], [6], [7]. Another approach is retrieval-augmented generation (RAG), which involves providing external documents to the model during the generation process, and having it connect its output to the available evidence [8], [9], [10], [11], [12]. Fact verification methods go just a little further. They not only enhance the generation, but also evaluate the generated claims against retrieved evidence and determine whether the claims are supported, contradicted, or insufficiently grounded [13], [14], [15]. In addition, uncertainty estimation and calibration techniques have been explored to quantify the reliability of the model and improve confidence in its reliability [16], [17], [18], [19]. Recently, hybrid strategies have started to integrate neuro-symbolic reasoning with the Small Language Model (SLM)-based verification. These methods seem to be promising as they might improve logical consistency without the need to repeat large-model verification as in [20], [21] and [22].

There have been some useful advances in these techniques, but they do not address the entire problem by themselves. Each approach, in many cases, targets only a single aspect of hallucination detection. Some detection methods, for instance, consider the overall response as one unit. This can be dangerous as an answer can be correct in most respects but with a single false statement. However, retrieval-based methods bring in external evidence, and depend on whether the retrieved documents are relevant and complete. Fact verification can provide a more cautious judgement, but is sometimes

expensive if each of the many claims in a lengthy answer has to be verified. Uncertainty estimation is also a helpful signal, but confidence alone does not make a claim factually true. The constraints indicate that there is a need for a more integrated pipeline for hallucination detection. A successful system will be able to break an answer into claims, support each claim with retrieved evidence, check the evidence efficiently, and resolve cases in which the evidence is ambiguous or conflicting by employing structured reasoning.

Inspired by this viewpoint, this paper introduces a hybrid hallucination detection and mitigation framework that combines different complementary components into a single architecture. The framework does not treat the answer in one continuous paragraph. Rather, it first subdivides the answer into atomic assertions, which can then be verified independently. From there, each claim is linked to the appropriate evidence by retrieval. A small lightweight language model would then serve as a verifier and tell if the claim is supported, contradicted, or remains ambiguous. If the verifier is not certain or the evidence looks contradictory, a neuro-symbolic reasoning layer is applied, which is designed to check with rules and reason contextually. Finally, the framework generates trust-aware outputs that are interpretable, fluent, and verifiable.

This work’s primary contribution is bringing retrieval grounding, atomic claim decomposition, SLM-based verification, and the neuro-symbolic reasoning together for hallucination detection and mitigation at the claim level. The framework is not just designed to look at the answer in its entirety, but to look at smaller claims within the answer. This facilitates the identification of partial hallucinations, particularly in answers that are basically correct but include one or two statements that unsupported statements. The small language model also makes the verification step is relatively light and simple, which is crucial if there are numerous claims to verify. In addition to classification, the reasoning layer also explains why a claim is accepted, rejected, corrected, or treated as insufficient. Last but not least, the framework generates a trust-aware output making the final response not only fluent but also more factually reliable.

The rest of the paper is arranged in the following way. Section 2 discusses the related work and positions the proposed work within existing hallucination detection research. Section 3 explains the methodology, including the claim decomposition, retrieval grounding, verification, and reasoning stages. Section 4 presents the experimental setup, results, and analysis. Section 5 discusses the main findings, limitations, and possible directions for future work.

2 Related Work

2.1 Hallucination in Large Language Models

Today, LLMs models are able to perform a variety of language tasks such as question answering, summarisation, dialogue, code generation, and so on. However, good language performance is does not necessarily mean factual correctness. These models can generate answers that sound convincing and read smoothly even if the information is incorrect, unsupported or even partially fabricated. This behaviour is usually described as hallucination [1], [23], [24]. The danger is further heightened when LLMs are used

in contexts where factual correctness is crucial, like in health guidance, educational institutions, or scientific research.

Hallucination doesn't typically originate from just one source. Some previous studies indicate that it can be caused by incomplete knowledge in the model, uncertainty in next-token prediction, weak reasoning over complex prompts, or the inconsistency between the model's internal knowledge and the external real world [1], [25]. Hallucinations can be generally classified into two categories. When the output contradicts the input or context given, it is considered intrinsic hallucination. In the second case, the model provides extra or inferred information, and is not supported by the evidence given which is known as extrinsic hallucination [1], [2]. These errors may appear in minor ways like a misplaced word or number, or at the sentence level or in multi-step reasoning.

Another challenge in controlling hallucination is that LLMs generate text sequentially, often selecting words that are statistically probable in a given context, without verifying whether each statement is supported by evidence [26]. This may result in fluent but inaccurate answers. A few studies also suggest knowledge overshadowing, where common patterns in the training data overwhelm correct but rare information [27]. Yet another issue is that a model can have factual knowledge embedded in it, but the knowledge is inconsistent when the model is used to produce an answer [28]. That is, just knowing a fact is not necessarily the reliable production of that fact for an LLM.

Empirical research reveals that hallucination is not restricted to any particular task. It seems to be present in summarisation, open domain question answering, dialogue systems and even code generation, but the nature of the error can vary across the different applications [25], [29]. In summarisation, a detail can be introduced in the summary that is not included in the source, and in question-answering, a confident answer can be given without any supporting evidence in the source. To facilitate such errors to be studied, benchmarks and evaluation frameworks have been introduced, including FACTSCORE, HalluLens, and HALOGEN. These resources are used in more detailed, fact-level evaluations, and enable comparisons of hallucination behaviour across models and tasks [30], [31], [32].

More recently, automatic detection of hallucination has been investigated. These approaches include self-consistent, internal-state analysis, probabilistic verification and multi-agent reasoning approaches [3], [4], [5], [6]. These methods are helpful, but they are not necessarily easy to apply in practice. Some of them involve multiple estimations, several retrieval steps, or verification steps that can be costly if the response is lengthy. Another issue is that detection, evaluation, and mitigation are considered three distinct activities. The separation can reduce the overall pipeline's practicality for use. A more integrated approach may prove to be helpful, in which grounding, claim verification, uncertainty estimation, and reasoning-based correction all contribute to enhancing the factual fidelity of LLM-generated outputs.

2.2 Hallucination Detection Methods

With the increasing use of LLMs in knowledge-intensive tasks, the detection of hallucination is a growing challenge. The problem does not have the same effect across all

settings. Some methods require labeled training data, some depend on external documents, and some require multiple calls to the model that can be time consuming and expensive. Due to these differences, the approaches that are available work in very different ways in practice.

Black-box (zero-resource) verification is one direction that is common. These methods attempt to identify hallucination but without inspecting the model itself. One well-known example is SelfCheckGPT. It produces multiple responses from the same model, and then checks for consistency among the answers. The idea is quite simple: factual statements should be more consistent across generations, and hallucinated statements more likely to vary from one response to another [5]. Other approaches rely on consistency checking from multiple perspectives, in which multiple responses or reasoning paths are checked for consistency with respect to a conclusion [4].

There is a line of work related to hallucination that is based on reasoning processes for the detection of hallucinations. For example, ChainPoll compares the similarity of the various reasoning paths that produce an output, and if they are similar, it suggests that the output may be reliable [6]. Multi-agent debate methods take a different approach. They ask several instances of a model to examine, challenge or criticize an answer, and if they disagree, then the response may contain hallucination [33]. These techniques are useful when the mistake has to do with incorrect logic and not an incorrect fact. This is a tradeoff, however, that the cost comes along. Including multiple reasoning paths or multiple agents typically involves additional inference time, potentially making these approaches less practical in low-latency systems.

The other one is to use signals coming from the inside the model. For internal-state-based detection, researchers examine internal states such as token probabilities or confidence patterns of the model’s output to determine the likelihood of hallucination during the generation process [3]. The benefit of this approach is that it does not necessarily entail a separate source of evidence or a retrieval process. At the same time, it typically relies on having access to model internals, which may not be available in many closed-source LLMs. It can also give predictions that sound unsure, but it doesn’t necessarily explain what in the answer is factually incorrect.

Retrieval-based detection is a different approach. These techniques evaluate the outputs against external resources like documents, knowledge graphs, or vector databases. For instance, GraphEval checks factual consistency using structured graph representations [13]. This type of grounding is helpful, particularly if the answer includes named entities, dates, or relations that can be compared with other sources. However, retrieval is not a complete solution. The verification step can become unreliable if the retrieved evidence is weak, noisy or narrow.

HalluLens and HALOGEN are benchmark datasets that enable a more systematic comparison of hallucination detection methods [31, 32]. In conjunction with these benchmarks, small language models (SLMs) have been investigated as lightweight verifiers for large models. The concept is not just theoretical: the goal is to use a smaller model to determine whether a claim is likely to be hallucinatory and then ask the smaller model to evaluate the output of the larger model. Previous studies have shown that SLMs can identify false claims with a reasonable level of accuracy

while reducing latency and deployment complexity compared to performing inference on large models multiple times [34].

Despite these advances, there are still some drawbacks. There are many existing approaches that still evaluate the overall response instead of the specific claims within a response. This can make it more difficult to detect partial hallucinations, as only part of the response may be correct, while an extra statement remains that isn't supported. Other methods require repeated sampling or multi-agent debate, or retrieval pipelines that do not necessarily provide complete evidence. These challenges point to the need for a more efficient, practical and claim-level, multi-stage detection system.

2.3 Retrieval-Augmented Generation for Factual Grounding

Retrieval-Augmented Generation (RAG) is one of the most common ways to improve factual reliability in LLM systems. Rather than depending only on knowledge stored in the model parameters, a RAG system retrieves relevant evidence during inference and uses that evidence to guide the generated answer [8], [9], [10]. In many cases, this helps reduce hallucination because the response is tied more closely to verifiable information.

In a typical RAG pipeline, the query is encoded, relevant documents are retrieved from a vector database or knowledge base, and the retrieved evidence is incorporated into the response generation [10]. Studies in this area show that RAG is particularly effective for knowledge-intensive tasks, where static model memory is insufficient [8].

Recent research has extended the basic retrieval pipeline using multi-hop retrieval and reasoning. For example, multi-hop RAG on knowledge graphs improves performance for queries that require connecting evidence from multiple sources [11]. Domain-specific RAG systems such as RAGClin and customized documentation QA frameworks show that targeted retrieval corpus can significantly improve factuality in specialized applications [35], [36].

A parallel research stream focuses on the quality and efficiency of retrieval. Methods such as ML-DPR improve dense passage retrieval through domain adaptation [37], while architectures such as Bauhaus optimize vector database infrastructure for large-scale LLM retrieval [38]. Other frameworks, including MEGA-RAG and MultiRAG, use multi-evidence aggregation and answer refinement to strengthen grounding across different evidence sources [12], [39].

Evaluation methods such as Face4RAG assess how well retrieved evidence supports generated outputs, enabling more systematic measurement of factual consistency in RAG systems [40], [41].

Despite these benefits, RAG does not completely resolve hallucinations. Retrieval may fail if relevant evidence is missing, poorly sequenced, or contains errors. Moreover, most RAG systems ground responses on the document level, while more detailed verification at the claim level remains limited. This suggests that the retrieval process should not be used alone, but should be combined with subsequent stages of verification and reasoning.

2.4 Fact and Claim Verification

Claim verification is a natural complement to retrieval-based grounding. While retrieval can provide potential evidence, verification determines whether the statements made are supported, contradicted, or insufficiently grounded.

Early fact-checking systems combined supervised classifiers with document retrieval that predicted whether evidence supported or refuted a claim [42]. Recent methods use LLMs and structured reasoning to verify claims requiring multi-hop evidence aggregation. For example, CRAVE applies contradictory reasoning to verify explainable claims [14], whereas human-inspired verification frameworks imitate the reasoning techniques used by expert fact-checkers [15]. Similarly, reasoning-path-based methods model the logical progression from evidence to conclusion for multi-hop verification [43].

Hybrid methods have also emerged. In particular, neuro-symbolic verification combines knowledge graphs, rules, or logical inference with neural language understanding to improve factual validation [20]. Systems such as MulCoB-MulFaV, for example, explore multimodal and multilingual fact verification, where claims may involve different languages or evidence types rather than only English text passages [44].

Another related approach is multi-agent fact checking. Here, several agents evaluate the same claim separately and their judgments are then combined [45]. The method may increase reliability as one agent could challenge and correct another agent’s conclusion. The downside is that it often requires more retrieval and inference on the retrieved evidence, particularly if each agent must reason over the retrieved evidence.

Even with these developments, many verification systems still judge outputs at a fairly coarse level. A response may be mostly correct but still contain one false statement, and that isolated error can be missed if the whole answer is evaluated together. These systems also depend on two fragile steps: whether the retrieval module finds the right evidence, and whether the reasoning module interprets that evidence correctly. This is why structured verification is likely to be more useful when it is combined with atomic claim decomposition.

2.5 Fine-Grained Factual Evaluation and Atomic Decomposition

One problem with many hallucination studies is that they judge the whole response at once. In practice, an LLM answer is rarely completely right or completely wrong. It may contain a correct definition, a wrong date, and an unsupported explanation in the same paragraph. This makes response-level evaluation too coarse for many cases.

FACTSCORE is an important example of work that moves toward a more fine-grained view. It compares the generated text with the original text to assess the factual accuracy of the generated text, based on atomic claims extracted from the original text [30]. This also illustrates how a response can appear sensible at a global level, but still contain a number of assertions that are either unsupported or even invented.

A related concept can be found in studies of attributed question answering, in which atomic fact decomposition helps parse the parts of the question before they are

sent to a verifier [46]. This provides more detail in the evaluation process and decreases the risk of missing partial hallucinations.

This direction has also been supported by several benchmarks and datasets. HalluLens and HALOGEN enable more systematic evaluation of across various models and tasks [31], [32]. AutoHall uses automatically generated factual and hallucinated examples to reduce the work required to build detection datasets [47]. A few fine-grained evaluations have been associated with retrieval-based systems. For Face4RAG, for example, generated claims are directly compared to retrieved evidence, making the verification process more evidence-aware [40].

The results of these studies indicate that atomic decomposition is not only an evaluation detail, but also a useful design choice for hallucination detection. If a system is shifted away from response-level judgment to claim-level checking, it can detect exactly where an answer becomes unreliable. This approach can be supported by a framework that decomposes the answers generated and then uses retrieval grounding and verification for each claim.

2.6 Uncertainty Estimation and Confidence Calibration

The primary challenge in managing hallucination is that LLMs can appear to be correct when they actually are not. When the users reads a polished answer, they may assume that the model “knows” what it is saying, but the confidence in the wording may not always match the accuracy of the information. That is why uncertainty estimation and confidence calibration have gained significance in the field of trustworthy LLMs.

Uncertainty estimation attempts to quantify the amount of confidence that can be placed in a model’s output. Uncertainty can come from a variety of sources in LLMs. The model might not know all the information that it needs about a subject. Other times, the prompt may contain ambiguity, or evidence may be weak. Epistemic uncertainty is typically distinguished from aleatoric uncertainty, which is associated with the ambiguity of the input, in previous surveys [16]. This distinction is important because there is a need for different methods of dealing with uncertainty in a trust-aware system.

The problem of confidence calibration is somewhat different from the above, as it deals with whether the model’s confidence is aligned with its actual correctness. A model that assigns high confidence should also be correct most of the time. In classification problems temperature scaling has been applied for a long time for calibration and in generative tasks it has recently been adapted to the generative domain [17]. LogitAC changes the logit’s behaviour to better calibrate the model and to identify out-of-distribution cases [18].

Abstention is also an idea that falls within this category. Selective prediction lets the model make predictions only when it is confident enough instead of answering all questions [48]. This can be helpful in situations where safety is a concern: it may be better to have an uncertain answer than a definite but incorrect one. Uncertainty signals within the internal representation of the output have also been used to determine whether the output is likely to contain errors [49].

However, it is not the same thing as a factual correctness. A model may still be wrong while remaining confident, if its internal knowledge is outdated or incomplete in its internal model. The problem is more difficult with long-form responses since one claim might be true while another is not. Therefore, uncertainty estimates are more helpful for the next steps in the process, including retrieval, claim verification, and reasoning, than for serving as a measure of reliability alone.

2.7 Neuro-Symbolic Reasoning for Reliable Verification

While LLMs may be able to follow patterns and generate reasoning explanations, they may not always be logically consistent. Neuro-symbolic reasoning can be beneficial here. It brings symbolic knowledge such as rules, knowledge graphs or formal constraints to guide the reasoning process in addition to neural language understanding.

The neural model is typically used in a neuro-symbolic system to process the language part of the problem, which includes tasks such as understanding the statement or comparing it with evidence. Then, a more controlled layer of reasoning is added—this is the symbolic part. For instance, one rule might be that the claim has to be supported by the evidence a numerical claim. This is particularly true when it comes to fact checking, where an answer must seem plausible and also make sense in relation to the evidence.

There has been recent research showing the benefits of using language models in combination with symbolic logic for factual validation in conversational AI [20]. In this sense, knowledge graphs can be very informative, as they provide structured representations of entities and relations, and can facilitate multi-step reasoning [50]. Meanwhile, other research studies demonstrate symbolic reasoning capabilities in LLMs, though not consistently or formally, as indicated in [51].

Strategies to alleviate this weakness include structured reasoning frameworks which attempt to steer outputs toward valid reasoning paths [52]. In addition, entangled reasoning architectures enable LLMs to access and interact with a knowledge graph during the inference process, which seems promising for enhancing factual consistency [53].

The main disadvantage is the difficulty of integration and cost. Neuro-symbolic techniques can increase interpretability and logical control; however, they may prove more difficult to implement in real-time systems. Nevertheless, they establish a useful foundation for verifying hallucinations, particularly when combined with retrieval grounding and simple, lightweight verification models.

2.8 Small Language Models for Efficient Verification

Because large language models are computationally expensive and difficult to implement for repeated verification, researchers have become increasingly interested in small language models (SLMs) as efficient verification modules.

SLMs use fewer parameters, which makes them faster, lighter, and easier to calibrate for specialized tasks. Prior studies show that, through techniques such as

distillation, fine-tuning, and task-specific optimization, SLMs can achieve strong performance in targeted verification settings [21]. For this reason, they are attractive for real-time hallucination detection pipelines.

A particularly promising design is to use the LLM as the generator and the SLM as the verifier. In this setup, the SLM evaluates generated outputs to detect factual inconsistencies, contradictions, or unsupported claims. Research on hallucination detection using SLM shows that this approach can achieve competitive performance at much lower computational cost than repeated large-model inference [34]. Similarly, SLM Meets LLM shows that combining a lightweight verifier with a large generator can improve latency, interpretability, and consistency [22].

Compared with larger models, SLMs can be easier to use for repeated verification, ensemble checking, and uncertainty-based filtering. They are lighter, faster, and more practical when many claims need to be checked. The trade-off is that they usually have less broad world knowledge. Because of that, an SLM may miss subtle factual errors if it has to judge a claim without supporting evidence. This makes retrieval grounding important. In practice, SLMs are likely to work best as verifiers when they are paired with retrieved evidence and structured reasoning, rather than used as standalone fact-checkers.

2.9 Trustworthy and Explainable AI

With LLMs becoming more functional, it is not just about the performance. A system can respond very promptly and accurately, but the user must determine whether the response is reliable. This is particularly important in high-stakes situations, where a black-box response can be hard to accept. When a model provides a medical explanation, makes a policy recommendation, or summarizes of a scientific paper, the user should be able to see why the model produced that particular answer, and whether it is evidence-based.

In natural language processing, explainable AI (XAI) aims to make the behavior of AI models easier to understand. Some methods highlight important input words, illustrate possible reasoning paths, or provide explanations after the model generates an answer [54]. This is especially important for LLM systems, as their hallucinated responses can appear fluent and confident. If a polished response is not supported, it may be accepted by a user without being explained or verified.

The key to trustworthy AI is more than just explanation. It also entails reliability, fairness, accountability, and the ability to question and verify the output. In the scientific domain, errors from LLM responses can be detrimental to subsequent analysis or decision-making, as seen in the case of SciTrustc [55] and similar problems. At a broader level, reliable artificial intelligence research indicates that when the need arises, model outputs should be evidence-backed, confidence-backed, and subject to human scrutiny.

Despite this, explanation is still difficult. While a number of explanation techniques are able to provide some hints about the behavior of a model, they do not fully explain the internal reasoning of large neural models. In addition, there is a conflict between performance and interpretability, in that a model can perform well on a benchmark while still providing shallow or incomplete explanations. This motivates a structure in

which generation is linked with verification, calibration and reasoning that users can follow more transparently.

To provide a comprehensive overview of existing approaches, Table 1 summarizes representative methods for hallucination detection and mitigation in large language models, along with their key contributions and limitations.

As summarized in Table 1, existing research considers hallucinations from a variety of individual perspectives, such as detection, retrieval grounding, verification, uncertainty estimation, and reasoning. However, these components are rarely integrated into a coherent, claim-level, and computationally efficient framework.

2.10 Research Gap and Motivation

Existing literature demonstrates significant progress has been made in the areas of hallucination analysis, detection, retrieval grounding, claim verification, uncertainty estimation, neuro-symbolic reasoning, and SLM-based verification. However, these areas of research are still largely conducted independently.

Hallucination research explains the problem well, but it does not provide a coherent solution for practical application. Detection methods such as self-consistency checking and internal-state analysis can detect unreliable output, but many of them operate at the response level and may fail to capture partial hallucinations. Retrieval-augmented generation improves grounding, yet false claims can still get through due to retrieval failures, faulty evidence, and document-level grounding. Fact verification methods provide stronger validation, but they are often computationally heavy and not tightly integrated with efficient generation pipelines. Uncertainty estimates and confidence scores are useful because they can show when a model’s output should be treated with caution. But they do not actually establish evidence of truth or falsity. Neuro-symbolic methods are used to achieve logical consistency and make the reasoning process easier to interpret, but many of them are not easily integrated and can be costly in practice. Another potential benefit of SLM-based verification is that it provides a less burdensome means of verifying generated claims. In most of the current work, however, SLM verification is not directly linked with retrieval grounding or structured reasoning. An important challenge remains: To design, a claim-level hallucination detection framework that can utilize retrieval, atomic decomposition, lightweight verification, and reasoning without being too costly to use.

This study contributes by proposing of a hybrid framework for hallucination detection and mitigation that aims to address this gap. The first step in the framework is to decompose the generated outputs into atomic claims. It then maps each claim to external retrieval, verifies the claim by using a lightweight small language model, and refines claims that are uncertain or conflicting using neuro-symbolic reasoning. This design aims to address multiple practical concerns: the ability to detect partial hallucinations, to verify claims against evidence, to keeping the verification process computationally manageable, and to make the final decision easily explainable. The proposed framework will enable more reliable and comprehensible LLM-based systems in real-world use.

Table 1 Summary of Hallucination Detection Methods

Category	Representative Studies	Main Contribution	Main Limitation
Hallucination Detection (Self-Consistency)	SelfCheckGPT [5], Multi-perspective consistency [4], InterrogateLLM [7]	Detects hallucinations via consistency across multiple generations without external knowledge	Requires repeated sampling; lacks fine-grained claim-level analysis
Reasoning-Based Detection	ChainPoll [6], Multi-agent debate [33]	Identifies unreliable outputs through reasoning disagreement and multi-agent critique	Computationally expensive; unsuitable for real-time deployment
Internal-State-Based Detection	Internal state analysis [3]	Uses hidden states and token probability patterns for real-time detection	Requires access to model internals; limited interpretability
Retrieval-Augmented Generation (RAG)	RAG survey [8], MEGA-RAG [12], MultiRAG [39], RAGClin [35]	Grounds outputs using external evidence to improve factual accuracy	Sensitive to retrieval quality; noisy or missing evidence can propagate errors
Fact and Claim Verification	CRAVE [14], Human-like verification [15], Multi-agent fact-checking [45]	Verifies claims through structured reasoning and multi-hop evidence	Computationally expensive; often applied post-generation
Fine-Grained Evaluation (Atomic Claims)	FACTSCORE [30], Atomic decomposition [46], AutoHall [47]	Evaluates factual consistency at the claim level using atomic decomposition	Primarily evaluation-focused; limited integration into full pipelines
Uncertainty Estimation & Calibration	Uncertainty survey [16], Temperature scaling [17], LogitAC [18], [48], Abstention [48]	Quantifies confidence and mitigates overconfident predictions	Does not directly ensure factual correctness
Neuro-Symbolic Reasoning	Hybrid verification [20], KG-guided reasoning [50], Entangled reasoning [53]	Combines neural models with symbolic reasoning for logical consistency	Complex integration; increased computational overhead
Small Language Model (SLM) Verification	SLM-based detection [34], SLM Meets LLM [22]	Enables efficient and low-latency verification using lightweight models	Limited world knowledge; depends on external grounding
Explainable & Trustworthy AI	XAI in NLP [54], SciTrust [55]	Improves transparency, interpretability, and trustworthiness of outputs	Explanations may be shallow and insufficient for full reliability

3 Proposed Methodology

3.1 Problem Formulation and Notation

LLM outputs may be grammatically sound and well-organized, but they are not necessarily accurate. A response can contain assertions without evidence, minor inaccuracies, or even statements that contradict credible sources. Therefore, the goal of this work is to detect and mitigate hallucination on a more granular level, by examining the claims within the generated answer, instead of the entire answer itself.

Let the user query be represented as Q . Given this query, the LLM first produces an initial answer A_0 :

$$A_0 = LLM(Q) \quad (1)$$

At this stage, A_0 is treated as an unverified response. Instead of accepting it directly, the framework decomposes the answer into a set of atomic factual claims:

$$F = \{f_1, f_2, \dots, f_n\} \quad (2)$$

Here, each f_i denotes a small, self-contained claim extracted from A_0 . For each claim f_i , relevant supporting evidence is retrieved from a document collection $\mathcal{D}$ using a semantic retrieval function:

$$E_i = \text{Retrieve}(f_i, \mathcal{D}) \quad (3)$$

where E_i consists of the top- k evidence passages along with similarity scores.

A lightweight verification module (Small Language Model, SLM) then evaluates the relationship between the claim and the retrieved evidence, producing a probabilistic assessment:

$$G_i = \{y_i, p_i^+, p_i^-, u_i\} \quad (4)$$

where:

- $y_i \in \{SUPPORTED, CONTRADICTED, UNDER - SPECIFIED\}$ is the predicted label
- p_i^+ is the probability that the claim is supported
- p_i^- is the probability that the claim is contradicted
- u_i represents uncertainty

Subsequently, a neuro-symbolic reasoning layer refines this decision using logical rules and contextual analysis:

$$R_i = \{\hat{f}_i, v_i, w_i\} \quad (5)$$

where:

- $\hat{f}_i$ is a corrected or validated claim
- $v_i \in \{SUPPORTED, CORRECTED, REJECTED, INSUFFICIENT\}$ is final verdict
- w_i is a natural-language explanation

Finally, the system aggregates the outcomes across all claims to generate a trust-calibrated response:

$$A^* = \text{Aggregate}(\{R_i\}_{i=1}^n) \quad (6)$$

along with an overall trust score indicating the reliability of the output.

The main symbols used in the proposed framework are defined as follows. Let Q denote the user query and A_0 denote the initial answer generated by the LLM. The generated answer is decomposed into a set of atomic claims F , where each individual claim is represented by f_i . The document corpus used for evidence retrieval is denoted by $\mathcal{D}$, and the retrieved evidence for the i -th claim is represented as E_i . The output of the SLM verification gate is denoted by G_i , while the final reasoning output produced after neuro-symbolic reasoning is denoted by R_i . Finally, the trust-calibrated final response generated by the framework is represented as A^* .

3.2 System Overview

The proposed framework performs claim-level hallucination detection and mitigation through a structured multi-layer verification pipeline. Instead of trusting the confidence of the LLM itself, the system first breaks a generated answer into smaller factual claims. The individual claims are then verified using evidence retrieval, probabilistic verification, and reasoning-based refinement. This design is helpful because hallucination may often be present only in a small part of an otherwise correct answer, and a one-stage detector might fail in these cases.

Figure 1 shows the overall architecture of the framework. The LLM generates an initial response A_0 to a given query Q . This response is not considered final. It is broken down into atomic claims, each of which goes through three stages: semantic grounding, uncertainty-aware verification, and neuro-symbolic reasoning. The outputs of these layers are then synthesized to generate a trust-calibrated response.

Initial answer generation: After receiving a query, the LLM produces a natural language answer from its learned knowledge. While this response could be well presented and helpful, it might also contain information that is unsupported, incorrect, or based on assumptions or unreliable evidence. For this reason, the generated answer is used only as a starting point for further checking.

Atomic fact decomposition: The initial response is broken down into smaller factual chunks, which can be checked individually. Atomic claims are individual statements that may, in principle, be substantiated with external evidence. This change transforms the task from determining the correctness of an entire response to checking the correctness of each claim, making it easier to notice partial hallucinations.

Layer-1 Semantic grounding: The system uses vector-based semantic search to find relevant evidence in a document corpus for each claim. The retrieval step returns evidence passages and similarity scores, quantifying the match between the retrieved content and the claim. This layer helps minimize reliance on the LLM’s internal knowledge and provides the later verification stages with a fact-based foundation for decision-making.

Layer-2 Uncertainty and consistency gate: The retrieved evidence and the claim are sent to a small language model (SLM). The SLM determines whether the

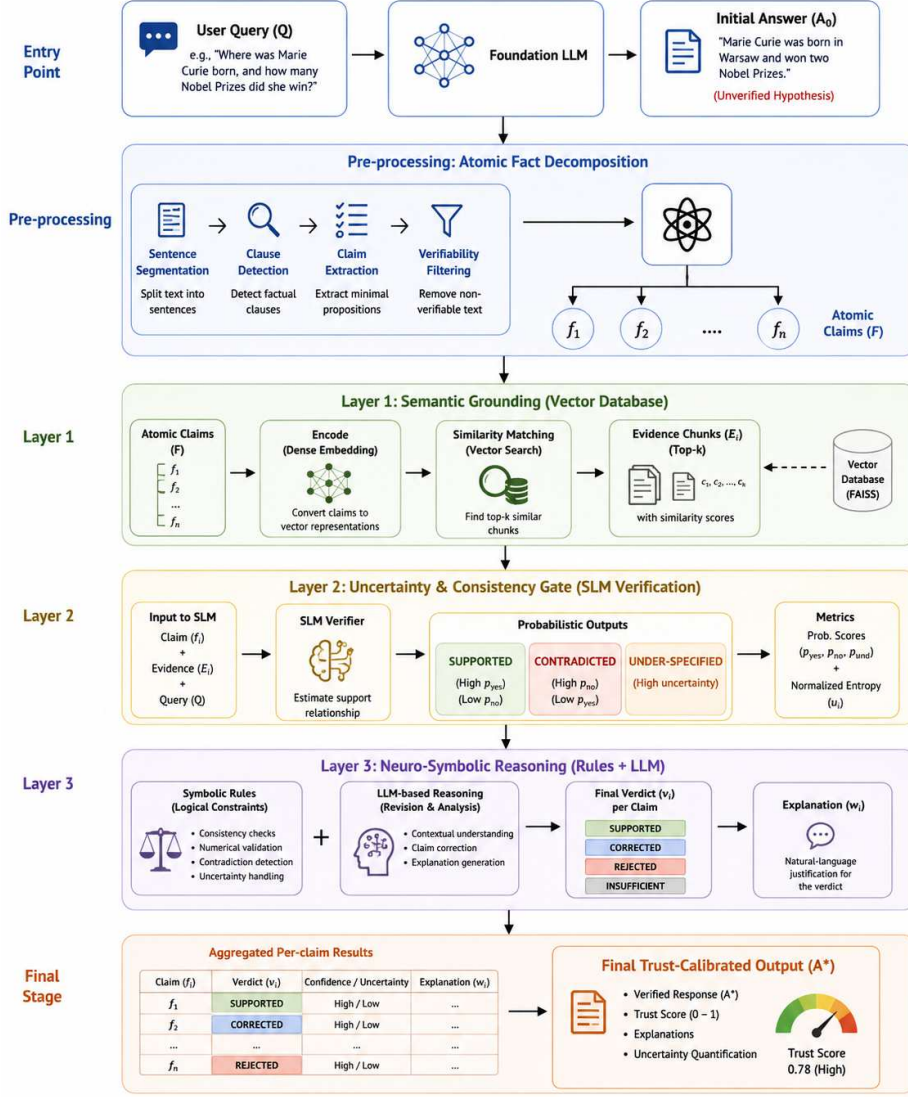


Fig. 1 Proposed multi-layer framework.

evidence supports the claim, contradicts the claim, or is insufficient. Along with the label, it also produces confidence and uncertainty values. This gate can be used to handle claims that are not confident enough to be classified to be classified as either supported or rejected, rather than categorizing every claim as supported or rejected.

Layer-3 Neuro-symbolic reasoning: In some cases, even after SLM verification, the claims may still remain uncertain. In those cases, the reasoning layer is used to perform rule-based checks and contextual reasoning in order to make a more refined decision. It can confirm supported claims, reject contradicted claims, or update claims

in light of evidence that clearly shows that they are inaccurate. This layer also helps to make the decision process easier to interpret as it gives reasons for the final decision.

Trust-calibrated output: Once all claims are processed, the system merges the claim-level judgments into the overall response A^* . Supported claims are retained, correctable claims may be updated and unsupported claims may be deleted or identified as unreliable. The system also generates a global trust value showing the level of trust in the final answer. In this way, the output is not only rewritten from the original LLM response, but also reconstructed from verified and evidence-based statements.

Overall, the framework has a number of practical advantages. It allows for fine-grained verification since every factual statement is judged separately. It retrieves evidence to inform decisions, rather than relying only on the LLM’s internal knowledge. It also includes uncertainty estimates for more cautious handling of ambiguous cases. The reasoning layer provides interpretability, and the compact verifier makes the verification process more tractable than repeated large-model checking.

3.3 LLM Generation (Initial Response)

Given a user query Q , the framework begins by asking an LLM to generate an initial answer. This step works like a normal AI assistant: the model reads the query and produces a natural language response. The response is denoted as A_0 . At this stage, however, no factual checking has been applied. The answer may come from the model’s learned knowledge, or it may also use optional query-level context when such context is available. Formally, this generation step can be written as $A_0 = LLM(Q, C_q)$, where C_q represents optional contextual information retrieved from external sources or previous interactions. If no extra context is provided, the model generates the answer only from the query.

LLMs are good at producing fluent and coherent text across many domains. That fluency, however, can be misleading. The model may include unsupported statements because of incomplete knowledge, biased training patterns, or overgeneralization from similar examples. Its internal confidence is also not a reliable guarantee of factual correctness.

For example, consider the question: “Where was Marie Curie born, and how many Nobel Prizes did she win?” An LLM might answer: “Marie Curie was born in Warsaw and won two Nobel Prizes.” This answer is fluent and appears factually reasonable. Still, the framework does not treat it as a trusted fact immediately. It treats the response as an initial, unverified answer that must pass through the later claim-level verification stages. Even when a generated answer seems correct, it is not immediately accepted. Instead, it is sent to the next validation pipeline for claim-level analysis. The purpose of this stage is not to produce a perfectly correct answer, but rather to provide potential content for further verification. This framework assumes that hallucination can occur at this stage and therefore does not accept the output of the LLM’s as final. Instead, the generated text is sent to the Atomic Fact Decomposition module, which extracts individual claims for independent validation. This design separates generation and verification, allowing the system to detect and correct erroneous statements without limiting the expressiveness of the LLM.

There are several benefits to using a powerful generative model at the beginning of the pipeline. It enables the production of natural and human-like responses, allowing the system to interact effectively across diverse contexts. This generation step is useful because it allows the framework to handle complex and open-ended queries, not only short fact-based questions. It also reflects how LLMs are normally used in practice: first, the model produces an answer, and only after that can the answer be checked. In this framework, generation and verification are deliberately kept separate. The LLM is allowed to generate a natural response, while the later stages examine whether the factual claims in that response are actually reliable.

By treating A_0 as an initial answer rather than a final one, the framework gives the later verification layers a chance to assess the response before it is shown as trusted information. The output of this stage is a textual answer:

$$A_0 = \{s_1, s_2, \dots, s_m\} \quad (7)$$

where each sentence s_i may contain one or more factual claims. These claims are extracted and evaluated in the next stage.

3.4 Atomic Fact Decomposition

The initial LLM response A_0 may contain several factual statements, sometimes spread across multiple sentences. Checking the whole response as one block can hide smaller errors. For example, an answer may contain one correct sentence and one unsupported claim, but a response-level judgment may fail to separate the two. To avoid this problem, the framework first decomposes the generated text into independent atomic facts.

In this work, an atomic fact refers to the smallest self-contained statement that can be checked against external evidence. It may be verified as supported, contradicted, or insufficiently grounded. Formally, the generated answer can be represented as $A_0 = \{s_1, s_2, \dots, s_m\}$. The decomposition step then produces a set of factual claims $F = \{f_1, f_2, \dots, f_n\}$, where each f_i represents a single verifiable proposition.

The decomposition module looks for clauses that contain factual claims, named entities, numerical values, or definitional statements. Non-verifiable content, such as opinions, instructions, or purely stylistic writing, is excluded. For example, the answer “Marie Curie was born in Warsaw and won two Nobel Prizes” can be separated into two claims: “Marie Curie was born in Warsaw” and “Marie Curie won two Nobel Prizes.” Each claim can then be checked independently against external evidence. Let $\mathcal{D}(\bullet)$ denote the decomposition function. Then,

$$F = \mathcal{D}(A_0) \quad (8)$$

Each atomic fact f_i is expected to satisfy three conditions: it should be self-contained, meaning that it can be understood without relying heavily on surrounding text; it should be verifiable, meaning that it can be evaluated against external evidence; and it should be minimal, meaning that it should not contain multiple independent claims that need to be checked separately.

Let m denote the number of sentences in A_0 , L denote the total length of the text in tokens or words, and k denote the number of extracted propositions. Sentence

segmentation and clause extraction are performed over the input text, so the time complexity is approximately linear with respect to the input length, expressed as $T(L) = O(L)$. During execution, the algorithm stores the sentence list S and the extracted claim set F . Therefore, memory use also grows linearly with the input size, giving a space complexity of $S(L) = O(L)$.

The decomposition module may be implemented through rule-based parsing, dependency analysis, or a lightweight language model. In this framework, a hybrid strategy is used to balance accuracy and efficiency. This step does not require external evidence, so its computational cost is relatively low compared with retrieval or model inference stages.

Algorithm 1 Atomic Fact Decomposition

Require: Generated answer A_0

Ensure: Set of atomic facts F

```

1: Initialize  $F \leftarrow \emptyset$ 
2: Split  $A_0$  into sentences  $S = \{s_1, s_2, \dots, s_m\}$ 
3: for each sentence  $s \in S$  do
4:   Identify factual clauses within  $s$ 
5:   Extract candidate propositions  $P = \{p_1, p_2, \dots\}$ 
6:   for each proposition  $p \in P$  do
7:     if  $p$  is self-contained and verifiable then
8:       Add  $p$  to  $F$ 
9:     end if
10:  end for
11: end for
12: return  $F$ 

```

Atomic decomposition is useful because it turns hallucination detection into a claim-level task. Instead of asking whether the full response is correct or incorrect, the framework checks each factual statement separately. This makes partial hallucinations easier to detect, especially when the overall answer appears mostly correct. It also improves interpretability, since each verification decision can be traced back to a specific claim. Another benefit is targeted correction: unsupported or incorrect claims can be revised without changing the parts of the answer that are already valid.

Without this decomposition step, a single false statement may either invalidate the whole response or remain hidden inside a longer answer. The module therefore produces a structured set of claims, $F = \{f_1, f_2, \dots, f_n\}$, which are passed individually to the semantic grounding layer for evidence retrieval.

3.5 Semantic Grounding via Vector Retrieval (Layer-1)

After the generated response is decomposed into atomic facts $F = \{f_1, f_2, \dots, f_n\}$, each claim needs to be checked against external knowledge. In this framework, that step is handled by semantic retrieval. The purpose is simple: for every claim, the system tries to find evidence passages that are semantically close to it. These passages are later

used by the verification gate to decide whether the claim is supported, contradicted, or insufficiently grounded.

The knowledge corpus is first divided into smaller document chunks:

$$\mathcal{K} = \{c_1, c_2, \dots, c_M\} \quad (9)$$

where each chunk c_j is converted into a dense vector representation:

$$v_j = \mathcal{O}(c_j) \quad (10)$$

Here, $\mathcal{O}(\bullet)$ denotes the embedding function. In the same way, each atomic fact f_i is encoded as:

$$u_i = \mathcal{O}(f_i) \quad (11)$$

To estimate how closely a claim matches a document chunk, cosine similarity is used:

$$\text{sim}(f_i, c_j) = \frac{u_i \bullet v_j}{\|u_i\| \|v_j\|} \quad (12)$$

For each claim f_i , the retrieval module selects the top- k chunks with the highest similarity scores:

$$E_i = \text{Top}K_{c_j \in \mathcal{K}}(\text{sim}(f_i, c_j)) \quad (13)$$

The strongest retrieved match for a claim is represented by the maximum similarity score:

$$\text{sim}_{\max}(f_i) = \max_{c_j \in E_i} \text{sim}(f_i, c_j) \quad (14)$$

The final output of this layer is a structured evidence set:

$$E_i = \{(c_{i1}, s_{i1}), (c_{i2}, s_{i2}), \dots, (c_{ij}, s_{ij})\} \quad (15)$$

where s_{ij} denotes the similarity score for the retrieved chunk c_{ij} . This evidence set provides the factual context for the next stage. If the retrieved passages closely match the claim, the verification gate has a stronger basis for making a decision. If the evidence is weak or only loosely related, the claim may need to be treated more cautiously.

In the case of the claim “Marie Curie was born in Warsaw”, the retrieval module might return paragraphs like “Marie Curie was born in Warsaw, which was then known as the Kingdom of Poland” with a high similarity score. This type of evidence provides a clear semantic grounding for the claim and allows the next level of verification to assess whether the statement is supported, contradicted, or insufficiently specified.

Let n denote the number of facts, M the number of chunks in the knowledge base, and d the embedding dimension. In a naïve implementation, the system computes similarity between each fact and all available chunks, resulting in the following time complexity of $T = O(n \bullet M \bullet d)$. In practice, approximate nearest neighbor (ANN) indexing structures (e.g., FAISS) reduce complexity to approximately $T \approx O(n \log M)$. In terms of storage, the system maintains embeddings for all knowledge base chunks and temporary candidate lists for each fact. The storage requirements

Algorithm 2 Semantic Grounding via Vector Retrieval

Require: Atomic facts $F = \{f_1, f_2, \dots, f_n\}$, knowledge base K , top- k

Ensure: Evidence sets $\{E_1, E_2, \dots, E_n\}$

```
1: Precompute embeddings for all chunks in  $K$ 
2: for each fact  $f_i \in F$  do
3:    $u_i \leftarrow \text{Embed}(f_i)$ 
4:   Initialize candidate list  $C_i \leftarrow \emptyset$ 
5:   for each chunk  $c_j \in K$  do
6:      $v_j \leftarrow \text{embedding of } c_j$ 
7:      $s_{ij} \leftarrow \text{cosine\_similarity}(u_i, v_j)$ 
8:     Add  $(c_j, s_{ij})$  to  $C_i$ 
9:   end for
10:   $E_i \leftarrow \text{top-}k \text{ elements of } C_i \text{ by score}$ 
11:   $\text{sim}_{\max}(f_i) \leftarrow \text{maximum score in } E_i$ 
12: end for
13: return  $\{E_1, E_2, \dots, E_n\}$ 
```

can be expressed as $O(Md)$ and $O(k)$, respectively. Overall, the memory usage is dominated by the storage of embedding representations, resulting in a space complexity of $S = O(Md)$.

Semantic grounding plays an important role in this framework because it gives the verifier something external to compare with the generated claim. Self-consistency methods mostly depend on the model’s own outputs, which can be useful but may still repeat the same mistake. Retrieval-based grounding, in contrast, checks the claim against evidence from outside the language model. In general, a high similarity score suggests that the retrieved passage is closely related to the claim, although it does not automatically prove that the claim is true. A low score may indicate that the evidence is missing, weak, or only loosely connected. These signals are passed to the next layer, where uncertainty estimation and reasoning can make a more careful decision. For each fact f_i , Layer-1 produces:

$$G_i^{(L1)} = \{f_i, E_i, \text{sim}_{\max}(f_i)\} \quad (16)$$

The output contains the original claim, the retrieved evidence set, and the highest similarity score. These values are then used by the Uncertainty and Consistency Gate for probabilistic verification. Depending on the dataset, evidence may come from an indexed corpus, or it may already be included as contextual support in the prepared sample.

3.6 Uncertainty and Consistency Gate (Layer-2: SLM Verification)

Once evidence has been retrieved for an atomic fact, the framework passes the claim and its evidence to a lightweight Small Language Model (SLM). The SLM then checks whether the evidence supports the claim, contradicts it, or leaves it unclear. This layer

acts as a probabilistic verification gate that transforms semantic evidence into structurally valid conclusions. Unlike large generative models, The SLM is optimized for classification-style reasoning, which enables effective and understandable verification. For each fact f_i , the SLM receives:

$$X_i = \{f_i, E_i, Q\} \quad (17)$$

where:

- f_i = atomic claim
- E_i = retrieved evidence set
- Q = original user query (optional)

The evidence set is typically concatenated into a context sequence:

$$C_i = \text{Concat}(c_{i1}, c_{i2}, \dots, c_{ik}) \quad (18)$$

The SLM estimates class probabilities over three verification labels:

$$\dagger = \{SUPPORTED, CONTRADICTED, UNDER - SPECIFIED\}$$

Formally,

$$P_i = [p_i^{\text{sup}}, p_i^{\text{con}}, p_i^{\text{und}}] = SLM(f_i, C_i) \quad (19)$$

The predicted label is obtained by:

$$\widehat{\mathcal{Y}}_i = \arg \max_{\mathcal{Y} \in \dagger} p_i^{\mathcal{Y}} \quad (20)$$

For decision calibration, the framework derives: $p_{yes} = p_i^{\text{sup}}$, $p_{no} = p_i^{\text{con}}$. These values quantify agreement or disagreement between the claim and evidence. Uncertainty reflects the confidence of the prediction. We compute normalized entropy:

$$u_i = -\frac{1}{\log|\dagger|} \sum_{\mathcal{Y} \in \dagger} p_i^{\mathcal{Y}} \log p_i^{\mathcal{Y}} \quad (21)$$

Properties:

- $u_i \approx 0$: confident prediction
- $u_i \approx 1$: high uncertainty

To avoid overconfident errors, the system applies threshold-based gating:

$$G_i = \begin{cases} SUPPORTED, & p_{yes} \geq \tau_s \\ CONTRADICTED, & p_{no} \geq \tau_c \\ UNDER - SPECIFIED, & \text{Otherwise} \end{cases} \quad (22)$$

In practice, the decision threshold is selected empirically through threshold-sensitivity analysis. Although the optimal F1-score is achieved at a lower threshold (≈ 0.40), a higher threshold ($\tau \approx 0.6$) is used in the final system to enforce conservative verification and reduce false positive SUPPORT predictions.

For example, consider the claim “Marie Curie won two Nobel Prizes.” If the retrieved evidence clearly states that she received the Nobel Prize in both physics and chemistry, then the SLM can determine a high support probability and a low contradiction probability. In such a case, the claim is labeled as SUPPORTED. On the other hand, if the evidence is only loosely related or does not directly confirm the statement, then the probability of support is low and the model may place the claim in the 'UNDER-SPECIFIED' category. This behavior helps the verification layer remain alert when the available evidence is incomplete or unclear.

Evidence contains explicit confirmation. The SLM may output $p^{\text{sup}} = 0.68$, $p^{\text{con}} = 0.32$, and $p^{\text{und}} = 0.00$, leading to the final decision $G_i = \text{SUPPORTED}$. In contrast, unrelated claims produce low support probability and high uncertainty.

Algorithm 3 SLM Verification Gate

Require: Facts F , evidence sets $\{E_1, E_2, \dots, E_n\}$, thresholds τ_s, τ_c

Ensure: Gate decisions $\{G_1, G_2, \dots, G_n\}$

```

1: for each fact  $f_i$  do
2:    $C_i \leftarrow$  concatenate evidence  $E_i$ 
3:    $p_i \leftarrow \text{SLM}(f_i, C_i)$ 
4:    $p_{\text{yes}} \leftarrow p_i[\text{supported}]$ 
5:    $p_{\text{no}} \leftarrow p_i[\text{contradicted}]$ 
6:    $u \leftarrow \text{entropy}(p_i)$ 
7:   if  $p_{\text{yes}} \geq \tau_s$  then
8:      $G_i \leftarrow \text{SUPPORTED}$ 
9:   else if  $p_{\text{no}} \geq \tau_c$  then
10:     $G_i \leftarrow \text{CONTRADICTED}$ 
11:   else
12:     $G_i \leftarrow \text{UNDER-SPECIFIED}$ 
13:   end if
14: end for
15: return  $\{G_1, G_2, \dots, G_n\}$ 

```

Let n denote the number of facts, L the input token length to the SLM, and d the model hidden size. The inference cost for processing each fact grows proportionally with the input length and model size, resulting in the following per-fact complexity of $O(Ld)$. The overall time complexity becomes $T = O(nLd)$. This cost is still more manageable than repeatedly using the original LLM for verification, because the verifier is much smaller. Memory use mainly comes from two sources: the SLM parameters and the input tokens processed during inference. Thus, the space requirement can be expressed as $S = O(P + L)$, where P denotes the number of model parameters. In practice, the model size is the dominant factor, while the input length adds only a linear cost.

This layer converts retrieved evidence into structured verification decisions. It serves three main purposes:

1. Detecting whether the evidence agrees with or contradicts the claim

2. Estimating confidence through probability scores
3. Identifying unclear cases through uncertainty

Using an SLM verifier helps keep the pipeline relatively lightweight, since the framework does not need to sample or query a large model repeatedly for every claim. For each fact, Layer-2 produces:

$$G_i^{(L2)} = \{f_i, E_i, G_i, p_{yes}, p_{no}, u_i\} \quad (23)$$

This output includes the claim, its retrieved evidence, the gate decision, support and contradiction probabilities, and the uncertainty score. The neuro-symbolic reasoning module then uses these values for final validation and correction.

3.7 Neuro-Symbolic Reasoning (Layer-3: Rules + LLM)

The SLM Verification Gate gives a probability-based decision for each claim, but that decision is not always enough on its own. In some cases, the retrieved evidence may be weak, the claim may be only partly supported, or the SLM may assign a label with high uncertainty. To handle these borderline cases, the framework uses a neuro-symbolic reasoning module. This layer combines rule-based checks with language-model reasoning so that the output from Layer-2 can be refined, corrected, or rejected when needed.

For each atomic fact f_i , Layer-3 receives the following information:

$$Z_i = \{f_i, E_i, G_i, p_{yes}, p_{no}, u_i\} \quad (24)$$

Here, f_i is the atomic claim, E_i is the retrieved evidence set, G_i is the gate label produced by Layer-2, p_{yes} and p_{no} represent the support and contradiction probabilities, and u_i denotes the uncertainty score.

The reasoning layer uses a predefined rule set $\mathcal{R}$ to apply domain-independent checks. These rules are simple but useful. For example, a claim with very low semantic similarity should not be accepted as supported. A numerical claim should require explicit evidence for the number. A claim with high uncertainty should be treated cautiously rather than accepted too quickly. The rule set can also check whether retrieved passages contradict each other. These constraints do not replace the SLM verifier; instead, they add an extra layer of control before the final verdict is produced. Formally, each rule can be written as a function $r_j : Z_i \rightarrow \{0, 1\}$, where $r_j \in \mathcal{R}$. A value of 1 means that the corresponding rule is satisfied, while 0 means that the rule condition is not satisfied.

When rules alone cannot determine the outcome, a reasoning LLM performs structured evaluation:

$$Reason(f_i, E_i, G_i) \rightarrow \tilde{f}_i, v_i, w_i \quad (25)$$

where:

- $\tilde{f}_i$ = possibly revised claim
- v_i = final verdict
- w_i = explanation

Layer-3 outputs one of four outcomes:

$$\mathcal{V} = \{SUPPORTED, CORRECTED, REJECTED, INSUFFICIENT\}$$

Interpretation:

- SUPPORTED = claim verified as true
- CORRECTED = modified version supported
- REJECTED = false or contradicted
- INSUFFICIENT = evidence inadequate

The final verdict is determined as:

$$R_i = \begin{cases} SUPPORTED, & G_i = SUPPORTED \wedge u_i < \tau_u \\ REJECTED, & G_i = CONTRADICTED \\ INSUFFICIENT, & G_i = UNDER - SPECIFIED \wedge u_i \geq \tau_u \\ CORRECTED, & LLM \text{ revision improves factual consistency} \end{cases} \quad (26)$$

where τ_u is an uncertainty threshold.

Consider the claim that “Marie Curie won three Nobel Prizes.” If the retrieved evidence consistently states that she won two Nobel Prizes, the Layer-2 verifier may identify the claim as contradicted. The neuro-symbolic reasoning layer can then refine this result by rejecting the original statement or creating a revised version, such as “Marie Curie won two Nobel Prizes.” In this way, the reasoning module not only classifies the claim as incorrect, but also assists in making sensible corrections if sufficient evidence is available.

Algorithm 4 Neuro-Symbolic Reasoner

Require: Layer-2 outputs Z_i for $i = 1, \dots, n$, rules $\mathcal{R}$

Ensure: Final reasoning results $\{R_i\}$

```

1: for each fact  $f_i$  do
2:   Apply rule set  $\mathcal{R}$  to  $Z_i$ 
3:   if  $G_i = SUPPORTED$  and  $u_i < \tau_u$  then
4:      $R_i \leftarrow (f_i, SUPPORTED)$ 
5:   else if  $G_i = CONTRADICTED$  then
6:      $R_i \leftarrow (f_i, REJECTED)$ 
7:   else
8:      $(\tilde{f}_i, v_i, w_i) \leftarrow \text{Reason\_LLM}(f_i, E_i, G_i)$ 
9:      $R_i \leftarrow (\tilde{f}_i, v_i, w_i)$ 
10:  end if
11: end for
12: return  $\{R_i\}$ 

```

Let n denote the number of facts, L_r the reasoning input length, and d_r the size of the reasoning model. The cost of applying the rule set scales linearly with both the number of facts and the number of rules, resulting in a complexity of $O(n|\mathcal{R}|)$. In the worst case, the reasoning module may need to process every extracted fact. If this happens, the cost of the reasoning model grows with the reasoning input length and the model size, giving a complexity of $O(nL_rd_r)$. When the rule-checking cost is included, the total time complexity can be written as $T = O(n(|\mathcal{R}| + L_rd_r))$. In practice, the cost is usually lower because the reasoning model is not called for every claim. It is mainly used for uncertain, contradicted, or borderline cases. The memory requirement can be expressed as $S = O(P_r + L_r)$, where P_r denotes the parameters of the reasoning model.

This hybrid design has a few practical advantages. The rule-based part makes the decision process more controlled, while the reasoning model helps handle cases that are not clear from probabilities alone. It can also explain why a claim is accepted, rejected, corrected, or left insufficient. When enough evidence is available, the module may revise an incorrect claim instead of simply removing it. This reduces blind dependence on the SLM’s probability scores and gives the final decision a clearer reasoning basis. For each claim, Layer-3 produces:

$$R_i = \{\tilde{f}_i, v_i, w_i\} \quad (27)$$

where $\tilde{f}_i$ is the validated or corrected claim, v_i is the final verdict, and w_i is the explanation. These results are later aggregated to generate the final response.

In short, Layer-3 works as a reliability safeguard. It checks the SLM decision, corrects false or weak claims when possible, and produces an interpretable verdict before the answer is finalized. This layer turns probability-based verification into factual judgments that are easier to use in high-trust applications.

3.8 Trust-Calibrated Output (Final Response Generation)

After each claim has been verified and, when needed, refined by the reasoning layer, the framework rebuilds the final answer from the validated information. Instead of returning the raw LLM output, the system produces a trust-calibrated output that reflects the reliability of the underlying evidence. The final layer receives the set of reasoning results $\mathcal{R} = \{R_1, R_2, \dots, R_n\}$, where each element is represented as $R_i = \{\tilde{f}_i, v_i, w_i\}$. contains:

- $\tilde{f}_i$ = validated or corrected claim
- $v_i \in \mathcal{V}$ = final verdict
- w_i = explanation

Optionally, the original answer A_0 may also be used as contextual guidance. Claims are grouped by verdict:

$$S_{\text{sup}} = \{i | v_i = \text{SUPPORTED}\},$$

$$S_{\text{cor}} = \{i | v_i = \text{CORRECTED}\},$$

$$S_{rej} = \{i | v_i = REJECTED\},$$

$$S_{ins} = \{i | v_i = INSUFFICIENT\}$$

The validated content is constructed primarily from supported and corrected claims:

$$F^* = \bigcup_{i \in S_{sup} \cup S_{cor}} \tilde{f}_i \quad (28)$$

Rejected and insufficient claims are excluded from the factual summary. A scalar trust score quantifies overall reliability. Let,

- $p_i = p_{yes, i}$ = SLM support probability
- u_i = uncertainty
- w_i = importance weight (optional)

A weighted formulation is:

$$T = \frac{\sum_{i \in S_{sup} \cup S_{cor}} w_i p_i (1 - u_i)}{\sum_{i=1}^n w_i} \quad (29)$$

If equal weighting is used:

$$T = \frac{\sum_{i \in S_{sup} \cup S_{cor}} p_i (1 - u_i)}{n} \quad (30)$$

For interpretability, the score can be mapped to qualitative levels:

$$Confidence(T) = \begin{cases} High, & T \geq 0.8 \\ Moderate, & 0.5 \leq T < 0.8 \\ Low, & T < 0.5 \end{cases} \quad (31)$$

The final response A^* is generated from the claims that survive the verification and reasoning stages. In practice, this means the system tries to do four things: combine the validated claims into a coherent answer, keep the response aligned with the original query, avoid unsupported information, and include explanations when they are useful.

Formally, the final response is written as:

$$A^* = Generate(Q, F^*) \quad (32)$$

where F^* represents the set of supported or corrected claims used to build the final answer.

For example, consider the query: "Where was Marie Curie born?" If the reasoning layer verifies the claim "Marie Curie was born in Warsaw" as SUPPORTED, the final response may simply state: "Marie Curie was born in Warsaw." Along with this answer, the framework can also provide a trust score, which gives the user a rough indication of how reliable the verified output is.

Let n denote the number of facts and L_g denote the length of the generated output. The aggregation step checks the verified claims once, so its cost grows linearly with the number of facts, $O(n)$. The final response generation depends on the output length and the generation model size d_g , giving a cost of $O(L_g d_g)$. Combining these two

parts, the overall time complexity can be expressed as $T = O(n + L_g d_g)$. And also the space complexity as $S = O(n + L_g)$, where n accounts for storing the verified claims and L_g accounts for the generated response.

Algorithm 5 Trust-Calibrated Response Generation

Require: Reasoning results $\{R_i\}$, probabilities $\{p_i\}$, uncertainties $\{u_i\}$

Ensure: Final answer A^* , trust score T

```

1: Supported  $\leftarrow \emptyset$ 
2: score_sum  $\leftarrow 0$ 
3: weight_sum  $\leftarrow 0$ 
4: for each  $R_i$  do
5:   if  $v_i \in \{\text{SUPPORTED}, \text{CORRECTED}\}$  then
6:     Supported  $\leftarrow \text{Supported} \cup \tilde{f}_i$ 
7:     score_sum  $\leftarrow \text{score\_sum} + p_i(1 - u_i)$ 
8:   end if
9:   weight_sum  $\leftarrow \text{weight\_sum} + 1$ 
10: end for
11:  $T \leftarrow \text{score\_sum} / \text{weight\_sum}$ 
12:  $A^* \leftarrow \text{Generate\_Final\_Response}(Q, \text{Supported})$ 
13: return  $A^*, T$ 

```

This output layer helps prevent unsupported or incorrect claims from passing into the final answer. It also makes uncertainty more visible, rather than hiding it behind a fluent response. When the evidence is strong, the answer can be stated directly. When the evidence is weak or a claim has been corrected, the system can reflect that in the final response or in the explanation. This is useful in high-stakes settings, where users may need to know not only what the system answered but also how much confidence they should place in it.

The Trust-Calibrated Output layer completes the pipeline by turning verified claims into a coherent response with an estimated reliability score. In this way, the framework does more than detect hallucination; it also tries to return an answer that is grounded, explainable, and easier for users to evaluate.

4 Results and Analysis

4.1 Experimental Setup

To evaluate the proposed verification framework, we ran a set of controlled experiments covering both claim verification and question-answering settings. The aim was not only to check whether the system produced correct labels or answers, but also to see how reliable its confidence scores were, how well it handled calibration, and how much computational cost the pipeline introduced. For this reason, we used two benchmark datasets with different characteristics: SciFact for scientific claim verification

and HotpotQA for multi-hop question answering. SciFact gives a more structured verification setting, while HotpotQA is useful for testing the full pipeline in cases where an answer may depend on more than one piece of evidence.

Each input sample was processed through the complete multi-layer pipeline. In response to a user query or request, an initial response was generated through a large language model and then decomposed into atomic factual claims. These claims are first grounded through a vector-based retrieval module, and then verified using a small language model (SLM) that acts as a decision gate. Finally, a reasoning component applied lightweight logical constraints before producing a trust-calibrated output. For the SciFact task, each claim was given one of three labels, such as SUPPORTED, CONTRADICTED, or UNDER-SPECIFIED, while for HotpotQA, we evaluated both the correctness of answers and the consistency of verification across different pipeline configurations.

The verification layer is implemented using the Qwen2.5-1.5B-Instruct model, which employed as a lightweight probabilistic verifier rather than a generative model. Instead of relying on generated outputs, the decision-making process was based on token-level probabilities, where confidence is determined from the relative probability of positive versus negative responses. A decision threshold is applied to these probabilities to determine final labels. The threshold-sensitivity analysis showed that the best operating point was around $\tau \approx 0.40$. At this setting, the system achieved a more balanced trade-off between precision and recall.

To understand the role of each component, we also ran an ablation study. Specific parts of the pipeline were removed one at a time, including retrieval grounding, SLM-based verification, reasoning, and atomic decomposition. This made it possible to observe which components contributed most to the final performance and where the system became weaker when a layer was missing.

For claim verification, we used accuracy and macro-F1. For question answering, we reported Exact Match (EM) and token-level F1. Reliability was examined through ROC-AUC, precision-recall behaviour, and expected calibration error (ECE). Latency was also measured at both the component level and the full pipeline level, since a verification system is less useful if it becomes too slow for practical use.

The implementation used vector similarity search for retrieval and sentence-level embeddings for semantic matching. Experiments were kept under consistent settings, and results were averaged across runs to reduce the effect of random variation. This setup allowed us to examine not only whether the framework produced better answers, but also whether it handled uncertainty in a meaningful way and remained computationally practical.

4.2 Benchmark Performance

To examine how the framework behaves beyond a controlled verification setting, we evaluated it on both SciFact and HotpotQA. These two datasets test different parts of the system. SciFact focuses on scientific claim verification, while HotpotQA is more useful for observing answer quality in multi-hop question-answering cases. Together, they provide a broader view of the framework’s behaviour under different pipeline configurations.

The SciFact results are shown in Figure 2. The full framework achieved an accuracy of 0.511 and a macro-F1 score of 0.504. These values indicate moderate but stable verification performance across the claim labels. One interesting observation is that the no-reasoner variant produced almost similar results to the full framework. This may suggest that, for SciFact, the SLM verification gate does most of the classification work. The reasoning layer did not noticeably improve the overall accuracy or macro-F1 score in this structured setting. Even so, this does not mean the reasoning layer is unnecessary. Its value is more visible in explanation, correction, and ambiguous cases, which are not always captured well by aggregate classification metrics.

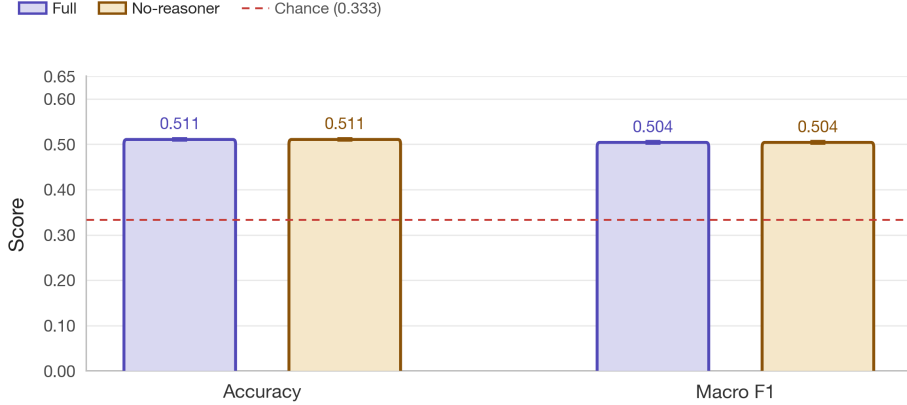


Fig. 2 SciFact benchmark results.

Figure 3 reports the HotpotQA results for the full pipeline, the retrieval-only baseline, and several ablation variants. The full pipeline achieved a token-level F1 score of 0.141 and an Exact Match score of 0.062. The retrieval-only baseline obtained a slightly higher token F1 score of 0.147 and a comparable Exact Match score of 0.066. At first, this may seem to suggest that retrieval alone is enough. A closer interpretation is that retrieval can give strong lexical overlap with the reference answer, while the additional verification stages make the system more conservative. This conservative behaviour may slightly reduce token overlap, but it is useful when the goal is not only to match the answer text but also to reduce unsupported claims.

The ablation results also show that the framework remains fairly stable when some individual components are removed. For example, removing the reasoning layer or the SLM verifier keeps the token F1 score in a narrow range of about 0.144 - 0.146. This suggests that these layers contribute more to verification behaviour, consistency, and interpretability than to large gains in standard answer-overlap metrics. Retrieval, however, is different. When the retrieval component was removed, token F1 dropped sharply to 0.034 and Exact Match decreased to 0.018. This result makes the role of semantic grounding very clear. Without relevant external evidence, the later verification and reasoning layers have very little reliable information to work with.

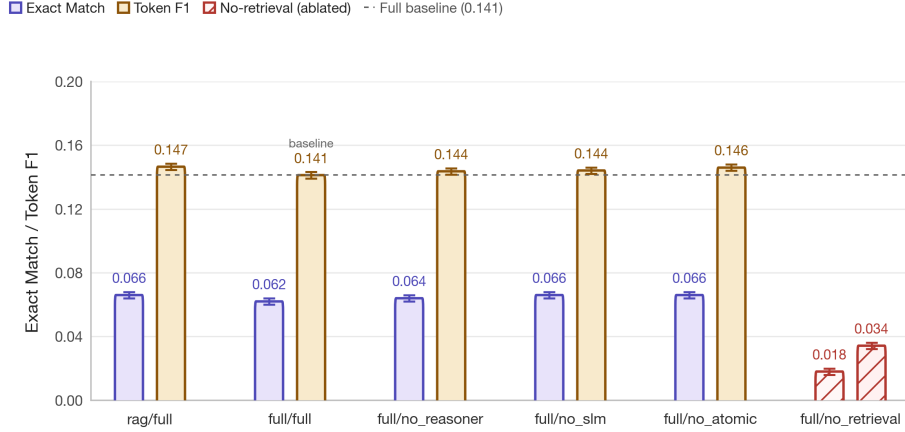


Fig. 3 HotpotQA performance comparison.

Overall, the benchmark results point to three main observations. First, the framework gives consistent results across both structured claim verification and open-domain question answering. Second, the SLM verification gate appears to be the main driver of SciFact classification performance, while the reasoning layer is more useful for explanation and refinement. Third, retrieval grounding is the most critical component for answer quality; without it, performance drops heavily. These findings suggest that the multi-layer design offers a practical balance between performance, reliability, and interpretability for hallucination detection and mitigation.

4.3 Ablation Study

To understand how much each component contributes to the framework, we ran an ablation study on HotpotQA. In this experiment, one module was removed at a time from the full pipeline. This made it possible to observe not only changes in answer quality, but also changes in the way the system made verification decisions.

Figure 4 shows how different ablation settings affected token-level F1 and answer-level performance. The full pipeline achieved a token F1 score of 0.141, which is used as the main comparison point. Removing the reasoning module, SLM verification layer, or atomic decomposition module did not cause a clear drop in token F1. In fact, these variants produced slightly higher scores: 0.144 for no-reasoner, 0.144 for no-SLM, and 0.146 for no-atomic.

This result may suggest that these components are not mainly improving surface-level lexical overlap with the reference answer. Their contribution appears to be more related to verification behaviour, interpretability, and cautious decision-making. Retrieval, however, shows a very different pattern. When the retrieval module was removed, token F1 dropped sharply to 0.034, which is about a 76% reduction from the full pipeline. This indicates that retrieval grounding is the most important component for answer quality in this setting. Without external evidence, the downstream verification and reasoning layers have little reliable information to work with.

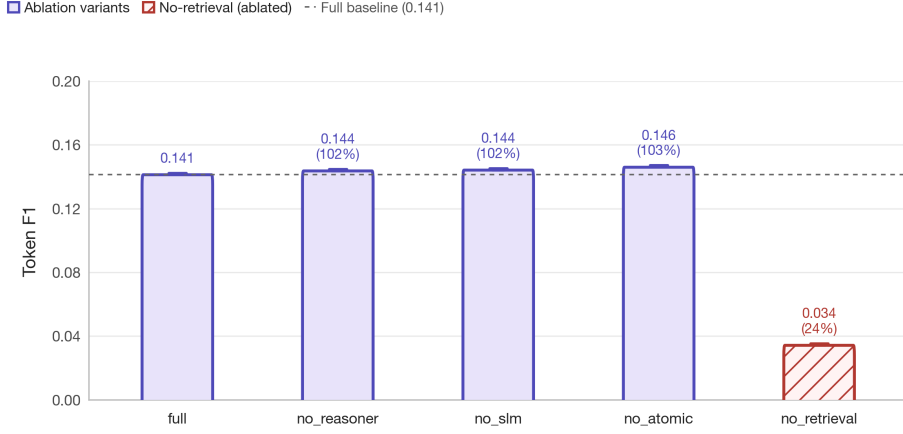


Fig. 4 HotpotQA ablation study.

To examine the internal verification behaviour more closely, Figure 5 shows the distribution of predicted verdicts across different variants: SUPPORTED, CONTRADICTED, and UNDER-SPECIFIED.

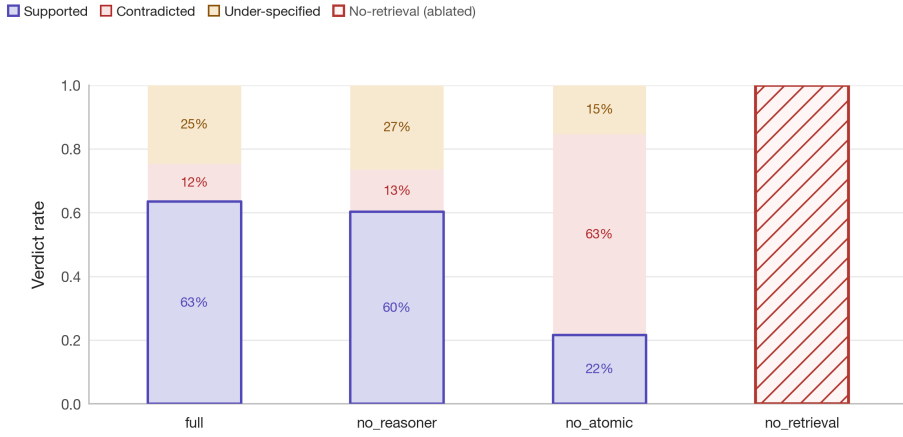


Fig. 5 Ablation verification profile.

The full model produced a relatively balanced profile, with 63% SUPPORTED, 12% CONTRADICTED, and 25% UNDER-SPECIFIED predictions. This suggests controlled verification behaviour rather than a strong bias toward one label. When the reasoning module was removed, the distribution changed only slightly: 60% SUPPORTED, 13% CONTRADICTED, and 27% UNDER-SPECIFIED. This supports the earlier observation that the reasoning layer has limited influence on aggregate

classification metrics, although it may still matter for borderline or explanation-heavy cases.

The no-atomic variant behaved quite differently. Without atomic decomposition, the proportion of CONTRADICTED predictions increased to 63%, while SUPPORTED dropped to 22%. This suggests that the system has difficulty interpreting mixed or compound statements when they are not separated into smaller claims. A long answer may contain several facts in one sentence; if they are evaluated together, the verifier may treat partial mismatch as a contradiction. The most extreme case appears when retrieval is removed. In that setting, the system almost collapses into the UNDER-SPECIFIED category, which is expected: without relevant evidence, the verifier has little basis for making confident support or contradiction decisions.

Overall, the ablation study shows that the framework depends on the interaction of several layers rather than a single module. Retrieval is the main driver of answer quality, but the other components shape how the system verifies, explains, and handles uncertain claims. The results also show why token-level F1 alone is not enough for evaluating hallucination detection. A component may have little effect on lexical overlap while still being important for reliable verification behaviour. This supports the need for a layered design that combines answer generation, evidence grounding, claim-level verification, and interpretable reasoning.

4.4 Verification Behavior Analysis

To understand the framework more clearly, we looked beyond the final performance scores and examined how it makes verification decisions. In particular, we analyzed how often the model assigns each verdict and where its predictions agree or disagree with the ground-truth labels. This helps show whether the framework is simply producing labels, or whether it is behaving in a cautious and meaningful way during verification.

Figure 6 shows the overall distribution of predicted verdicts. Among 1,618 verified claims, the model classified 690 claims (42.7%) as UNDER-SPECIFIED, 494 claims (30.5%) as CONTRADICTED, and 434 claims (26.8%) as SUPPORTED. The largest group is therefore UNDER-SPECIFIED. This suggests that the model often avoids making a strong support or contradiction decision when the evidence is weak or unclear. In a hallucination detection setting, this kind of caution can be useful. It is usually safer to mark a claim as uncertain than to incorrectly label an unsupported claim as supported.

The confusion matrix in Figure 7 gives a more detailed view of this behaviour. The framework separates SUPPORTED and CONTRADICTED claims with relatively low cross-confusion. Only 6.6% of supported claims are predicted as contradicted, and 7.5% of contradicted claims are predicted as supported. This indicates that, when the evidence is clear, the model can usually distinguish agreement from contradiction.

The more difficult cases appear around the UNDER-SPECIFIED class. A large portion of truly SUPPORTED claims, 36.1%, are predicted as UNDER-SPECIFIED. This lowers recall for the SUPPORTED class, but it is consistent with the conservative behaviour of the framework. Instead of accepting a claim when the evidence is not strong enough, the system often chooses uncertainty. At the same time, 48.0%

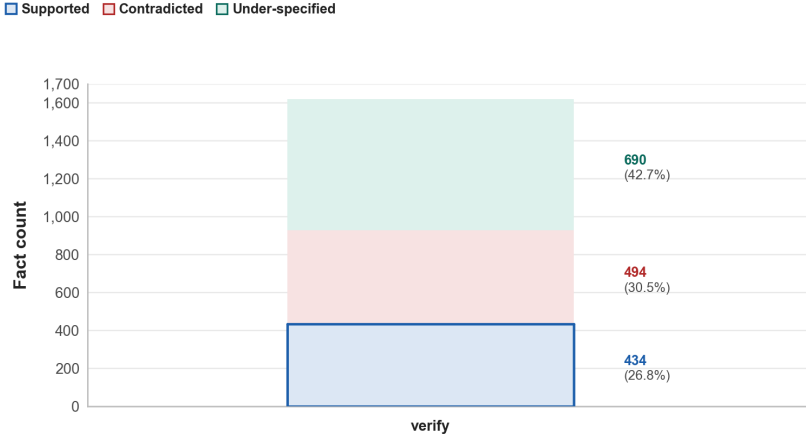


Fig. 6 Overall verdict distribution.

of UNDER-SPECIFIED claims are predicted as CONTRADICTED. This may suggest that weak, incomplete, or partially related evidence is sometimes interpreted as disagreement rather than lack of evidence. These borderline cases remain challenging because the difference between not enough evidence and evidence against the claim can be subtle.

Overall, the verification profile shows that the framework tends to favour caution over aggressive classification. Its main priority is to reduce incorrect SUPPORTED predictions, even if this means that some genuinely supported claims are moved into the UNDER-SPECIFIED class. This trade-off is important in real-world use, where a confident but wrong verification decision may be more harmful than an uncertain one. In that sense, the framework behaves in a trust-aware manner: it does not always give the most assertive label, but it tries to avoid validating claims that are not clearly supported by evidence.

4.5 Confidence and Calibration Analysis

Accuracy and F1 scores show how often the verifier is right, but they do not tell the whole story. In a verification system, the confidence of a prediction matters as well. If the model labels a claim as supported with high confidence, that confidence should ideally reflect the actual chance of being correct. To examine this behaviour, we analyzed both the confidence margin distribution and the calibration of the predicted probabilities.

Figure 8 shows the distribution of confidence margins. Here, the margin refers to the difference between the model’s support and rejection probabilities. A larger margin means the model has a stronger preference for one decision, while a smaller margin suggests that the evidence is less decisive. The histogram is concentrated mostly around higher margin values, roughly between 0.6 and 0.9. This indicates that many predictions are made with a clear preference between support and rejection. At the

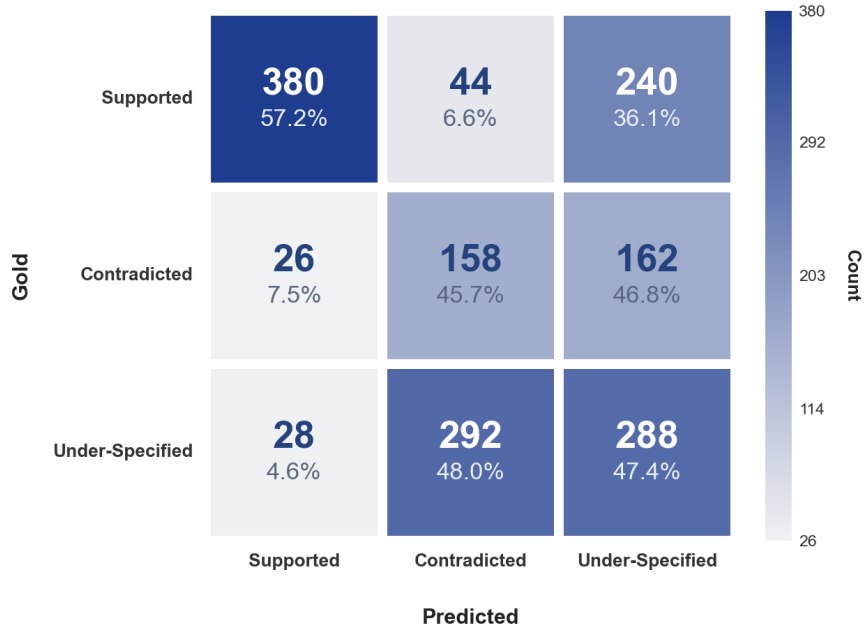


Fig. 7 Confusion matrix.

same time, there is still a visible group of samples below about 0.4. These are the more uncertain cases, where the model does not strongly favor either decision.

This pattern supports the use of a stricter decision threshold around $\tau \approx 0.6$. A lower threshold may accept more claims as supported, but it also increases the risk of accepting weakly grounded claims. A stricter threshold separates clearer decisions from borderline cases more cautiously. In other words, the model gives stronger confidence when the evidence is clear, while lower-margin cases are better treated as uncertain rather than forced into a supported label.

To check whether these confidence values are meaningful, we also examined the calibration curve in Figure 9. The reliability diagram compares predicted probability with observed accuracy. In an ideal case, the curve should follow the diagonal line, meaning that a prediction made with 70% confidence is correct about 70% of the time.

The calibration curve follows the diagonal reference line across most probability ranges. This suggests that the predicted support probabilities are reasonably aligned with empirical correctness. There are some small deviations at higher confidence values, where the model appears to overestimate correctness slightly. However, the deviation is limited and does not change the overall pattern. Higher predicted confidence generally corresponds to a higher observed support rate, which is the expected behaviour for a useful verifier.

Taken together, the confidence margin and calibration results suggest that the SLM gate produces informative confidence signals, not just class labels. This matters

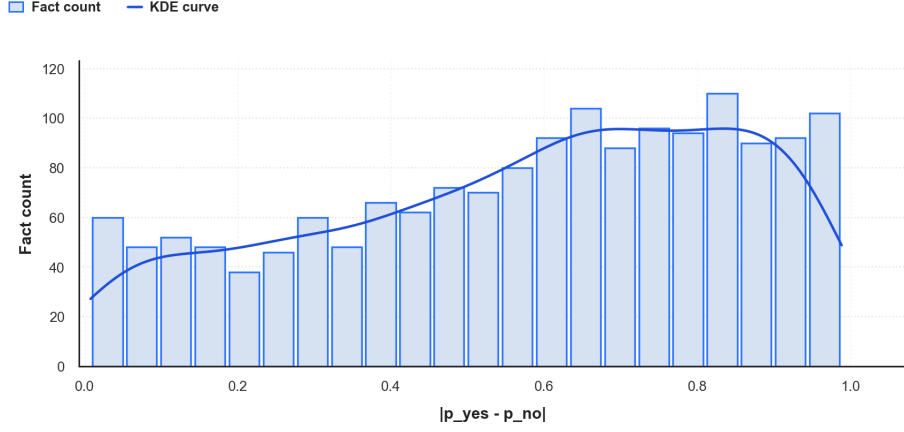


Fig. 8 Confidence margin distribution.

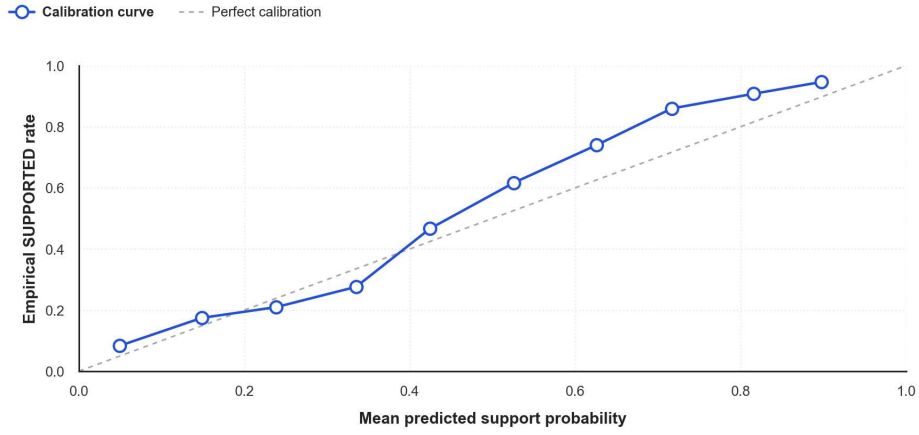


Fig. 9 Calibration curve.

for threshold-based verification, because the final decision depends directly on the probability values. When confidence is high and evidence is clear, the model can make a stronger decision. When confidence is low, the framework can handle the claim more cautiously. This supports the role of the SLM gate as an uncertainty-aware component within the full hallucination detection pipeline.

4.6 Threshold Sensitivity Analysis

To understand the verification layer more clearly, we examined how the system behaves under different confidence thresholds. In this framework, the final decision depends on the support probability p_{yes} . Changing the threshold directly changes how easily

the system accepts a claim as supported, so it is important to study how this affects precision and recall.

Figure 10 shows the threshold sensitivity curve. As the threshold increases, the F1-score first improves and reaches its highest value of 0.77 at around 0.40. After that point, the score gradually decreases. This pattern is expected in a threshold-based system. With a low threshold, the model accepts more claims as supported, which can improve recall but may also increase false positives. With a high threshold, the model becomes stricter. This can improve precision, but it may reject or mark some truly supported claims as uncertain.

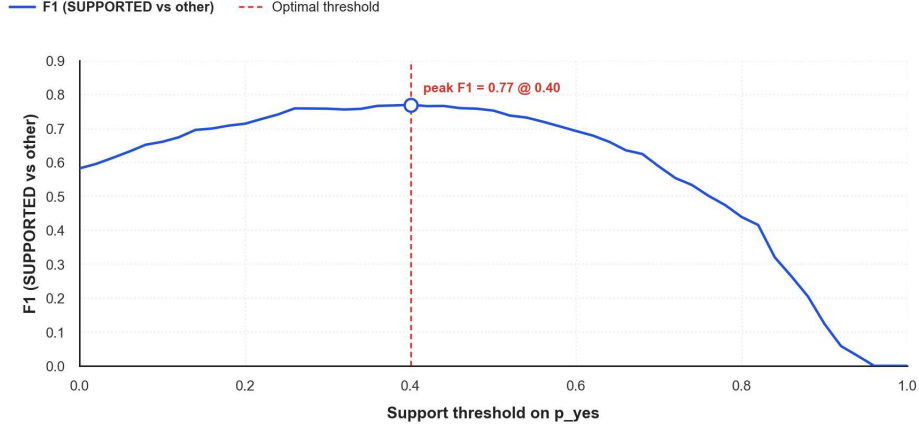


Fig. 10 Threshold sensitivity curve.

The curve suggests that $p_{yes} \approx 0.40$ is the best operating point if the goal is to maximize F1-score. Around this region, the model keeps a reasonable balance between accepting supported claims and avoiding incorrect support decisions. However, the final system uses a stricter threshold near ($\tau \approx 0.6$). This choice is intentional. In hallucination detection, a false SUPPORTED label can be more harmful than an uncertain label, especially when users may trust the verified output. A higher threshold makes the system more conservative and reduces the risk of accepting weakly grounded claims.

These results show that the framework can be adjusted depending on the application. A lower threshold may be useful when recall is more important, while a higher threshold is better when precision and reliability are the main priorities. For the final setting, we preferred the conservative option because it better matches the goal of reducing hallucinated or unsupported claims.

4.7 ROC and Precision–Recall Analysis

To examine the verification module from another angle, we also used Receiver Operating Characteristic (ROC) and Precision–Recall (PR) curves. These curves are useful because they do not depend on a single fixed threshold. Instead, they show how well

the verifier separates supported claims from unsupported or uncertain ones across different decision settings.

Figure 11 shows the ROC curve. The curve stays well above the diagonal reference line, which represents random guessing. This indicates that the verifier has a strong ability to separate positive and negative cases, with an AUC of 0.87. Across a range of thresholds, the model keeps a relatively high true positive rate while limiting the false positive rate. In practical terms, this means the verification layer is usually able to distinguish SUPPORTED claims from CONTRADICTED and UNDER-SPECIFIED cases when the evidence signal is clear.

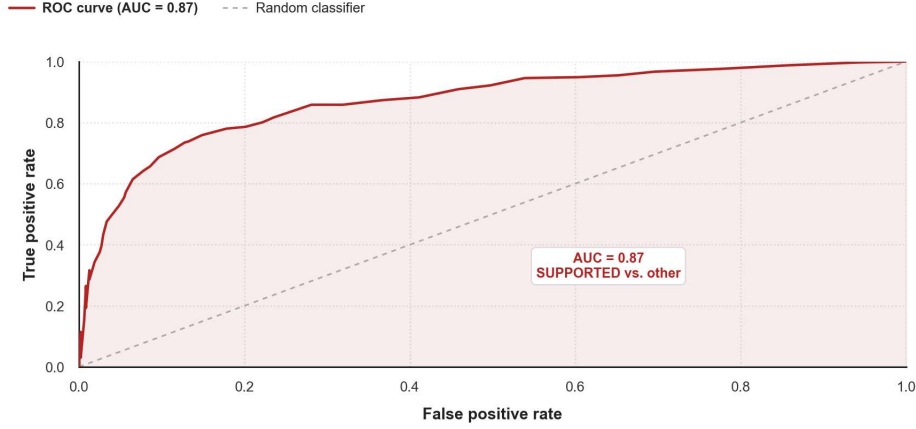


Fig. 11 ROC curve.

The Precision–Recall curve in Figure 12 gives a complementary view, especially because the class distribution is not perfectly balanced. The curve shows that precision remains reasonably high across much of the recall range, with an Average Precision (AP) score of 0.83. This is most visible in the middle recall region, around 0.3–0.7, where the model still identifies supported claims without introducing too many false positives.

As recall increases further, precision begins to decline, which is expected. At that point, the model starts accepting more borderline cases as supported, and some of those cases are likely to be weakly grounded or incorrect. This pattern is consistent with the threshold analysis: a more permissive setting can recover more supported claims, but it also increases the risk of accepting uncertain ones.

Taken together, the ROC and PR results suggest that the verifier provides useful separation across different operating thresholds. The ROC curve shows strong overall discrimination, while the PR curve gives a clearer picture of how precision behaves when the system tries to recover more supported claims. These results support the use of the verification layer as a probability-based decision component within the full framework.

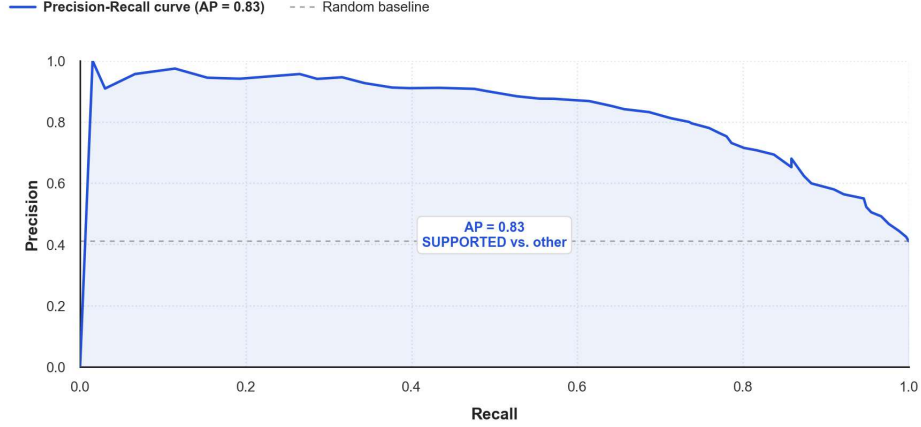


Fig. 12 Precision–recall curve.

4.8 Efficiency and Latency Analysis

Along with accuracy and reliability, we also measured how much time the framework needs to process an input. This is important because the system is built as a multi-layer pipeline. Even if a verification method performs well, it may be difficult to use in practice if each layer adds too much delay.

Figure 13 shows the latency breakdown across different pipeline settings. The full pipeline required about 20.6 seconds on average. Some ablation settings were slower: removing atomic decomposition increased latency to 49.8 seconds, while removing the reasoning layer resulted in an average latency of 23.3 seconds. The simpler variants were much faster. The no-retrieval and no-SLM settings required only 2.9 seconds and 2.2 seconds, respectively.

The stacked latency components show that SLM verification accounts for a large part of the total processing time when the full verification pipeline is used. This is expected, since model inference is more expensive than operations such as similarity search or rule checking. Retrieval adds comparatively little latency, which suggests that vector-based search remains efficient in this setup. The reasoning layer also adds only a small overhead, likely because it is applied selectively or through lightweight reasoning steps.

The stacked latency components show that SLM verification accounts for a large part of the total processing time when the full verification pipeline is used. This is expected, since model inference is more expensive than operations such as similarity search or rule checking. Retrieval adds comparatively little latency, which suggests that vector-based search remains efficient in this setup. The reasoning layer also adds only a small overhead, likely because it is applied selectively or through lightweight reasoning steps.

Overall, the framework shows a manageable balance between response time and verification quality. The full pipeline is slower than simplified variants, but the added cost mainly comes from evidence-based verification, which is central to the purpose

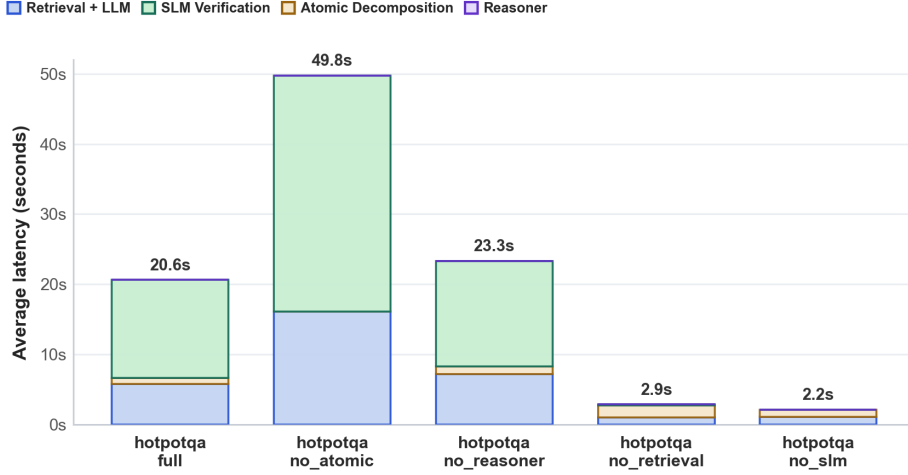


Fig. 13 Latency breakdown.

of the system. This suggests that the framework can be practical in settings where factual reliability and interpretability are more important than producing the fastest possible response.

4.9 Error Analysis

To better understand where the framework fails, we examined the error types rather than relying only on overall scores. This analysis compares the gold label with the model’s predicted label. It gives a more detailed view of the mistakes the system makes, especially in cases where the evidence is weak, incomplete, or difficult to interpret.

Figure 14 shows the error type distribution. Each bar represents one type of label transition, such as UNDER-SPECIFIED $\rightarrow$ CONTRADICTED or SUPPORTED $\rightarrow$ UNDER-SPECIFIED. This makes it easier to see which misclassification patterns occur most often.

The largest error category is UNDER-SPECIFIED $\rightarrow$ CONTRADICTED, with 292 cases, or 36.9% of all errors. This suggests that the model sometimes treats weak or incomplete evidence as if it directly contradicts the claim. In other words, when the evidence does not clearly support a claim, the verifier may become too negative instead of leaving the case uncertain. This is an important limitation, especially for borderline claims where partial evidence is present but not strong enough for a confident decision.

The second most common error is SUPPORTED $\rightarrow$ UNDER-SPECIFIED, with 240 cases, or 30.3%. Here, the system fails to accept some claims that are actually supported. This pattern matches the earlier confusion matrix and reflects the conservative behaviour of the verifier. The model often avoids giving a SUPPORTED label unless the evidence is strong enough. This helps reduce false support decisions, but it also lowers recall for the SUPPORTED class.

Another frequent error is CONTRADICTED $\rightarrow$ UNDER-SPECIFIED, which appears in 162 cases, or 20.5%. This means that some contradicted claims are treated

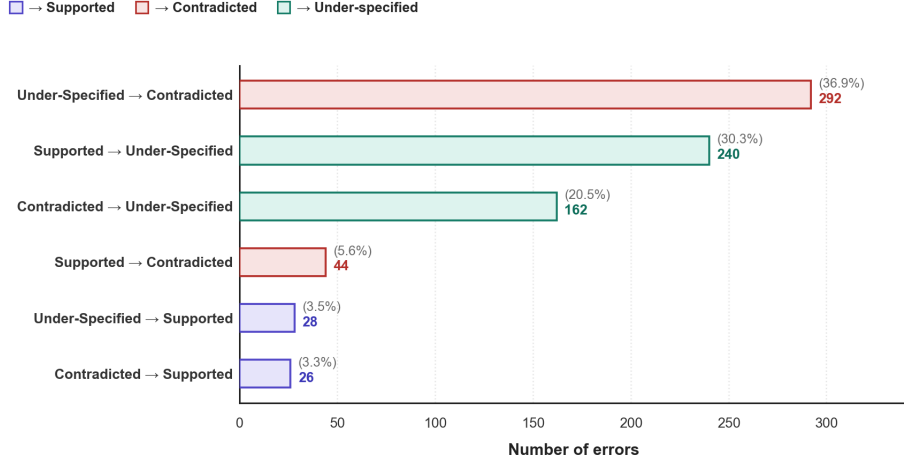


Fig. 14 Error type distribution.

as uncertain rather than being clearly rejected. This may happen when the retrieved evidence is relevant but not explicit enough, or when the contradiction depends on a small detail such as a date, number, or entity relation.

The less frequent but more serious errors are **SUPPORTED → CONTRADICTED** and **CONTRADICTED → SUPPORTED**. These occurred in 44 cases (5.6%) and 26 cases (3.3%), respectively. Although these mistakes are more concerning because they reverse the meaning of the label, their relatively low frequency suggests that the model rarely makes strongly incorrect confident decisions.

Overall, the error analysis points to three main observations. First, many errors are related to uncertainty handling rather than direct confusion between support and contradiction. Second, the framework shows a conservative tendency, often moving difficult cases into the **UNDER-SPECIFIED** category. Third, retrieval quality strongly affects verification quality. When evidence is missing, incomplete, or only loosely related, the model has difficulty separating uncertainty from contradiction. These findings suggest that future improvement should focus on better retrieval and clearer handling of the boundary between **UNDER-SPECIFIED** and **CONTRADICTED** claims.

4.10 Summary of Key Findings

The experimental results provide a more complete understanding of the performance of the framework under various conditions. However, SciFact had an accuracy of 0.511 and macro-F1 of 0.504. On HotpotQA, the framework achieved 0.146 token-level F1 score. The scores are not particularly high in absolute terms, but they demonstrate that the framework can be applied in a stable manner to two different types of tasks: structured claim verification and multi-hop question answering. This is important because it is not only one way of inputting information or one type of evidence for hallucination detection.

In addition, the ablation results demonstrate that the pipeline is not equally reliant on each component. The most important aspect is retrieval. The removal of it causes a sharp drop in performance, implying the relevance of evidence to the operation of the later layers of verification and reasoning. The SLM verification layer and the reasoning module do not always improve token-level F1 directly. Nevertheless, they seem to have different roles. They influence verification of claims, dealing with uncertainty and explaining decisions in the system. Their value in that regard is not completely captured by metrics of answer overlap, however.

The verification profile also indicates that the model has conservative behaviour. A lot of claims are categorized as UNDER-SPECIFIED when they have weak or ambiguous evidence, instead of being forced into SUPPORTED or CONTRADICTED. This will decrease recall for supported claims, but it will also decrease the risk of falsely validating an unsupported claim. This is a trade-off that is acceptable for the detection of hallucinatory experiences. An unconfident response is better than a confident one in many real world situations.

Results from the confidence and calibration analyses indicate the probability scores are useful is not too wide nor too narrow to support decision making based on thresholds. The highest F1 was at a threshold of approximately 0.40; the last system at a harder threshold of around $\tau \approx 0.6$. This is more due to the practical decision of precision rather than aggressive support prediction. The results of the calibration also show that the greater the confidence, the greater the correctness, but there is still a tendency to overestimate at high levels of confidence.

The ROC and Precision–Recall results corroborate each other. The verifier has a good capacity to distinguish between supported and unsupported or uncertain claims at different thresholds, with an AUC of 0.87 and average precision of 0.83. This is helpful since a verification system should not rely solely on a single decision point. The desired precision/recall ratio may vary according to applications.

Error analysis has more detailed information of the remaining weaknesses. The errors that occur are mostly related to a lack of certainty in handling, particularly in the area of UNDER-SPECIFIED and CONTRADICTED. Sometimes, when evidence has been retrieved that is incomplete or partially applicable, the verifier considers the absence of support as contradiction. Future enhancements to the system should include retrieval quality and the differentiation between weak evidence and evidence that directly contradicts a claim.

Overall, the findings indicate a practical balance between accuracy, reliability, interpretability and efficiency for the framework. Its ability to detect hallucination is not limited to just that, but also at the claim level with retrieved evidence, calibrated confidence, and reasoning-based refinement. This makes the system more appropriate for applications requiring reliability, as the user will receive outputs which are not only fluent, but also verifiable and trustworthy.

5 Discussion

5.1 Introduction

This paper introduces a multi-layer hallucination verification framework based on retrieval grounding, an SLM-based verification gate, and neuro-symbolic reasoning. The experiments investigate various elements of the framework such as detection performance, confidence behaviour, error patterns, and runtime costs. This section discusses what the results mean, how they compare with previous work, and what they indicate for practical applications. We also highlight main current drawbacks of the system and suggest possible avenues for future improvement.

5.2 Interpretation of Results

The results of the experiments indicate that the framework is able to distinguish SUPPORTED, CONTRADICTED and UNDER-SPECIFIED claims in different evaluation settings. The confusion matrix and per-class results demonstrate particularly good behaviour with CONTRADICTED claims for both precision and recall. When the evidence retrieved from the verification gate is an unambiguous refutation of a claim, then the gate is effective. The SUPPORTED class is different. It is a high precision (0.90) but recall is less. The model is typically right when it supports a claim, but it's not always right when we don't have enough evidence or it's ambiguous.

This is a characteristic of the conservative design of the framework. The system tends to end up in the UNDER-SPECIFIED category with weak or ambiguous cases. This helps to lower the number of false SUPPORTED predictions, but also misses some legitimate supported claims. This is a fair compromise for detecting hallucinations. In a real context, particularly in scientific or decision-making contexts, erroneous endorsement of an unconfirmed assertion can have more negative impact than uncertainty.

This is confirmed by the confidence analysis. The confidence score for a claim that is predicted as SUPPORTED tends to be higher, whereas the support probability for a claim that is not supported or contradicted tends to be lower. More significantly, retrieval similarity seems to be linked to the verification confidence. The more the evidence retrieved is semantically similar to the claim, the more likely the verifier is to form robust conclusions. This implies that the retrieval layer and the SLM verification gate are not being performed in isolation, but rather the quality of grounding influences the confidence and reliability in the verification step.

Also the threshold analysis indicates how important the final decision threshold is. The optimal F1-score is achieved at a moderate threshold, typically around 0.40 and strikes a balance between precision and recall. The final system, however, has a different threshold value close to $\tau \approx 0.6$. This selection will sacrifice some recall but decrease the probability of false SUPPORTED predictions. The calibration results also reveal that the scores obtained by the verifier are reasonably close to the proportion of correct answers, thus they are not just random numbers, but rather serve as a measure of their confidence in the answer and can be used in a threshold-based decision-making process.

The efficiency results indicate that the framework is practical with its multiple stages. The bulk of processing time is from SLM inference with relatively little overhead for retrieval. This is good as it means there is not a major bottleneck in the current arrangement. For many interactive verification scenarios, the response time may be usable for strict real-time deployment, although there is still room to optimize the system.

Another study that is useful in deriving the results from SciFact is the following: The complete framework and the no-reasoner variant are only slightly different, with 0.511 accuracy and 0.504 macro-F1. In other words, for SciFact, the reasoning layer makes less of a difference to the final classification score than retrieval and SLM verification. A likely reason is that many SciFact claims are already closely linked to short evidence passages, so that the verifier can make a decision with little further reasoning. The reasoning layer is not always necessary for explanation and if the reasoning is not a good fit for understanding the cases it is not clearly reflected in the overall statistics of this dataset.

5.3 Comparison with Previous Studies

The problem of hallucination detection in large language models has been widely studied, and existing approaches can be mainly divided into self-consistency methods, external fact-checking systems, small-model verifiers, and hybrid architectures. Our proposed framework is most closely compatible with hybrid and small-model-based approaches, and at the same time it achieves a closer integration between retrieval grounding, verification, and reasoning.

The SelfCheckGPT [5] detects hallucinations by measuring inconsistencies between multiple generated responses. Although this method is effective in zero-resource situations, it does not rely on external evidence and therefore faces problems with domain-specific factual verification. These methods primarily report consistency-based or ranking metrics rather than standard F1 scores, and their performance varies across datasets. In comparison, our framework achieves strong precision for SUPPORTED claims (approximately 0.90) by explicitly grounding verification in retrieved knowledge, which reduces the risk of confident but unverified predictions.

FACTSCORE [30] evaluates factual precision at the atomic level using web-scale retrieval and large language models. Although FACTSCORE demonstrates strong fine-grained factual evaluation performance using its own scoring metric, though it relies heavily on large models and external search. In contrast, our approach relies on a compact verifier model (Qwen2.5-1.5B) and a local vector database, which achieves competitive verification performance with considerable low latency (approximately 20–21 seconds for full pipeline processing, with significantly lower latency for ablated configurations), making it more suitable for real-time and resource-constrained settings.

Hybrid approaches such as SLM Meets LLM [22] suggest that smaller models can be useful verifiers for outputs produced by larger models. This is an important direction because repeated verification with a large model can quickly become expensive. Our results follow a similar trend. The framework achieves competitive verification performance while using a smaller verifier and operating without live web access. This

gives a useful trade-off: the system may not always outperform larger or more resource-heavy methods in absolute F1-score, but it remains more practical in settings where latency and computational cost matter.

Recent research on SLM-based verification [34] has reported F1 scores of approximately 0.55–0.60, demonstrating that it is possible to detect hallucinations in a lightweight manner. This is broadly consistent with our results. In particular, the system has improved with respect to the identification of CONTRADICTED claims by introducing semantic retrieval together with the SLM system, with a precision of 0.80. This implies that the verifier will gain from a source of evidence outside its own scope of knowledge, rather than evaluating the claim based on its internal evidence.

Knowledge-graph-based methods, for example, GraphEval [13] can achieve high accuracy on structured benchmarks because they rely on entity and relation information that has been curated. One problem is that building and maintaining knowledge graphs is not always easy, particularly as the target domain evolves. Our framework takes a lighter route by using vector-based retrieval instead. This facilitates adaptation to new collections of documents, while performance remains highly dependent on the quality of the retrieved evidence.

However, there are also methods that provide useful signals for hallucination detection that are based on uncertainty and probability, as described in [19]. In our example, the verifier achieves an AUC of 0.87, and the ROC and precision–recall curves demonstrate that it is able to distinguish supported claims from unsupported and uncertain ones at various thresholds. The calibration results also indicate that the confidence scores are not arbitrary in the sense that higher confidence is associated with higher correctness. This is crucial in verification environments where users must be aware of the label they are expecting as well as the level of trust they should have in this label.

As a whole, the comparison in Table 2 indicates that some of the previous systems obtain better absolute F1-scores, but they tend to use larger models or external search or structured knowledge resources. The proposed framework attains 0.511 accuracy and 0.504 macro-F1 with the pipeline being relatively lightweight, easily interpretable and calibration-aware. Based on these findings, the framework offers a viable solution for hallucination detection in environments where efficiency, transparency, and trustworthiness are crucial.

5.4 Implications of the Findings

The findings suggest a number of practical implications for detecting hallucinations in LLM systems. First, the strong precision of SUPPORTED and CONTRADICTED claims implies that the framework is effective in situations where the evidence clearly supports or clearly contradicts a claim. This is significant because the worst ‘hallucinations’ are the ones that are confident: answers that sound correct but aren’t backed up. When relevant evidence is available, this seems to be mitigated by the use of retrieval grounding and an SLM verifier. The second implication is the significance of semantic grounding. Confidence analysis reveals that there is a general trend of increased verification confidence as retrieval similarity increases. Simply put, if there is the system retrieves evidence that is very similar to the claim, then the verifier has a better chance of making a decision. This reinforces the notion that retrieval should not be something

Table 2 Comparison with Previous Hallucination Detection Methods

Method	Verification Strategy	Main Metric	Reported	Macro-F1	Micro-F1	ROC-AUC / AUROC	Avg. Latency
SelfCheckGPT [5]	Self-consistency sampling	AUC-PR up to 0.934		NR	NR	NR	NR
FACTSCORE [30]	Atomic fact checking + retrieval	FACTSCORE / error rate		NR	NR	NR	NR
ChainPoll [6]	Multi-step polling	AUROC 0.781		NR	NR	0.781	NR
GraphEval [13]	Knowledge-graph + NLI verification	Balanced accuracy up to 0.796		NR	NR	NR	NR
InterrogateLLM [7]	Query reconstruction consistency	AUC / B-ACC		NR	NR	varies by dataset	up to ~2 s
SLM LLM [22] Meets	SLM classifier + LLM reasoner	consistency/latency analysis		NR	NR	NR	NR
Proposed Framework	Atomic fact checking + Retrieval + SLM verification + neuro-symbolic reasoning	Macro/Micro-F1 AUC	+ 0.504	0.504	0.511	0.87	19.75 s

Note: NR (Not Reported) indicates that the metric is not available in the original publication.

that is merely added on. It is likely to be one of the key components of pipelines for hallucination mitigation, particularly compared to pipelines based on self-consistency or model self-reflection. Calibration results also play a role when deploying the system. Given the system’s confidence scores are reasonably in line with the observed correctness, they can be used to inform decisions at the downstream decisions. For instance, an application can identify supported claims with high confidence, contradictory claims, and claims that are subject to uncertainty that may be sent to humans for review. This type of confidence-aware behaviour is particularly beneficial in a situations such as scientific support, education or decision making, where a more uncertain response is better than a confident response that turns out to be incorrect. Finally, the latency analysis suggests that hallucination detection does not always require repeated calls to a large model or expensive multi-agent verification. The compact SLM verifier adds some processing time, but it keeps the pipeline more practical than approaches that repeatedly sample from a large LLM. Overall, the results suggest that a smaller verifier, when paired with retrieval and calibration, can provide a workable balance between reliability, interpretability, and computational cost.

5.5 Limitations

The proposed framework has several limitations that should be considered. The first one is related to the SUPPORTED class. Its precision is relatively high, but its recall is lower. This means the system is careful when it labels a claim as supported, but it also misses some claims that are actually correct. In many cases, these missed claims are moved into the UNDER-SPECIFIED category. This conservative behaviour helps reduce false positive support decisions, but it can also be too cautious when the evidence is sparse, indirect, or expressed differently from the claim. The second drawback comes from the retrieval phase. The framework relies significantly on the quality and coverage of the evidence corpus. The verifier might lack the context to make a sound decision if the relevant document is not found, poorly indexed, or retrieved with a low similarity. This may result in sub-optimal predictions or misclassification in certain cases. This is not a problem specific to this framework, but rather a challenge for retrieval-based verification systems. The other limitation is the evaluation context. While SciFact and HotpotQA offer some helpful benchmarks, they are not comprehensive enough to cover the scope of actual user queries. The questions can be ambiguous, casual, topic-specific, and/or include unstated assumptions that may not be directly substantiated. The present experiments provide a controlled perspective on the framework, but open-domain and interactive settings could lead to further difficulties. Lastly, the manually defined rule set embedded in the neuro-symbolic reasoning layer is still limited. The rules are intended to enhance the interpretability of the program and to provide a solution kept, but sentence cleaned for a few obvious cases, but they are not guaranteed to cover all reasoning patterns, or domain-specific constraints. The framework could benefit from more external knowledge sources, more elaborate domain rules, and adaptive rule-learning capabilities in later development to deal with more complex verification cases.

5.6 Future Research

The findings suggest a few avenues for future research. The first one is the recall of SUPPORTED claims. The framework is very cautious when assigning a supported label, but at the same time it may lead to some claims being assigned to the UNDER-SPECIFIED category when they are actually correct. Adaptive thresholding, confidence-aware re-ranking or multi-evidence aggregation can be explored in future versions to allow the construction of the support from multiple passages that are relevant rather than just one retrieved chunk. A further interesting direction is to evaluate the framework in more open-domain and multi-hop contexts. While the current configuration provides a clean view of the system, real user queries may not be as well formatted. A user might ask a broad question, merge multiple topics or anticipate linkages between information from different sources. In these situations, more intensive retrieval might be required. When the local evidence corpus is insufficient, structured knowledge graphs, more documents, or web-scale retrieval may be able to help. The neuro-symbolic reasoning layer could be enhanced as well. The rule set is currently manually defined so it is easier to interpret, but is limited in scope. Learned rules, hybrid neural-symbolic reasoning, and automatically adapted domain-specific constraints could be explored in future work. Feedback from humans and active learning can also help, particularly for cases that may be ambiguous for the system, where it is likely to get confused between uncertainty and contradiction. Finally, the framework should be assessed in additional domains, languages, and actual user interactions. This would provide a better understanding of the generalization of the approach to non-benchmark datasets and its applicability in realistic deployments.

5.7 Conclusion

This study proposed a multi-layered approach for hallucination detection and mitigation using semantic retrieval, SLM-based verification, and neuro-symbolic reasoning. The experiments demonstrate that the framework can reasonably and consistently separate SUPPORTED, CONTRADICTED, and UNDER-SPECIFIED claims in both claim verification and question-answering environments. The findings also indicate that the detection process does not need to rely on a large verifier model or multiple samples from an LLM. Even with a smaller verifier and retrieved evidence, combined with a simple set of reasoning constraints, the verifier can offer useful verification performance without sacrificing the computational tractability and interpretability of the system. The role of retrieval is particularly highlighted by the results of the ablations. If evidence grounding is not provided, the system cannot produce many correct answers or confident verification decisions. Structured claim-level classification is provided by the SLM verification gate, while the reasoning layer has a secondary role that is mainly related to explanation, correction, and uncertain cases. The framework also exhibits good confidence calibration, which is crucial for users to understand of the confidence associated not only with the label assigned by the system, but also the confidence that the label is correct. Some limitations remain. The system is still conservative with SUPPORTED claims and not every ambiguous claim is necessarily distinct from contradictions. Despite this, the results suggest the route of building a

trust-aware claim-level verification pipeline is a viable way to mitigate hallucination in LLM-generated responses. Future work will extend this work in several ways, including enhancing the retrieval quality, broadening the reasoning layer, and evaluating the framework for more domains and user interactions.

Statements and Declarations

Ethics Statement This study did not involve human participants, human data, animal subjects, or any clinical intervention. Therefore, ethical approval was not required.

Funding Declaration The authors declare that no funds, grants, or other support were received during the preparation of this manuscript.

References

- [1] L. Huang *et al.*, “A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions,” *ACM Trans. Inf. Syst.*, vol. 43, no. 2, Jan. 2025, doi: 10.1145/3703155.
- [2] S. M. T. I. Tonmoy *et al.*, “A Comprehensive Survey of Hallucination Mitigation Techniques in Large Language Models,” Jan. 2024, [Online]. Available: <http://arxiv.org/abs/2401.01313>
- [3] W. Su *et al.*, “Unsupervised Real-Time Hallucination Detection based on the Internal States of Large Language Models,” Jun. 2024, [Online]. Available: <http://arxiv.org/abs/2403.06448>
- [4] L. Kong, X. Zhong, J. Chen, H. Fu, and Y. Wang, “Multi-perspective consistency checking for large language model hallucination detection: a black-box zero-resource approach,” *Frontiers of Information Technology and Electronic Engineering*, vol. 26, no. 11, pp. 2298–2309, Nov. 2025, doi: 10.1631/FITEE.2500180.
- [5] P. Manakul, A. Liusie, and M. J. F. Gales, “SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models,” Oct. 2023, [Online]. Available: <http://arxiv.org/abs/2303.08896>
- [6] R. Friel and A. Sanyal, “Chainpoll: A high efficacy method for LLM hallucination detection,” Oct. 2023, [Online]. Available: <http://arxiv.org/abs/2310.18344>
- [7] Y. Yehuda, I. Malkiel, O. Barkan, J. Weill, R. Ronen, and N. Koenigstein, “InterrogateLLM: Zero-Resource Hallucination Detection in LLM-Generated Answers,” Aug. 2024, [Online]. Available: <http://arxiv.org/abs/2403.02889>
- [8] Y. Ma *et al.*, “A Survey of Retrieval-Augmented Generation (RAG) for Large Language Models,” in *2025 International Conference on Trustworthy Big Data and Artificial Intelligence (ICTBAI)*, IEEE, Aug. 2025, pp. 7–13. doi: 10.1109/ICTBAI68361.2025.00008.

- [9] S. AboulEla, P. Zabihitari, N. Ibrahim, M. Afshar, and R. Kashef, “Exploring RAG Solutions to Reduce Hallucinations in LLMs,” in *2025 IEEE International systems Conference (SysCon)*, IEEE, Apr. 2025, pp. 1–8. doi: 10.1109/SysCon64521.2025.11014810.
- [10] H. Tekgöz, H. Erdoğan, H. Uz, and H. Altınok, “Evaluation of Vector Database and LLM Models in Retrieval-Augmented Generation (RAG) Systems,” in *2025 10th International Conference on Computer Science and Engineering (UBMK)*, IEEE, Sep. 2025, pp. 572–576. doi: 10.1109/UBMK67458.2025.11206778.
- [11] M. R. A. Hamid, S. A. El-Regaily, and M. M. Aref, “Can Large Language Models Perform Retrieval-Augmented Generation as Multi-Hop Reasoning Over Knowledge Graphs?,” in *2025 International Mobile, Intelligent, and Ubiquitous Computing Conference (MIUCC)*, IEEE, Sep. 2025, pp. 360–365. doi: 10.1109/MIUCC66482.2025.11196859.
- [12] S. Xu, Z. Yan, C. Dai, and F. Wu, “MEGA-RAG: a retrieval-augmented generation framework with multi-evidence guided answer refinement for mitigating hallucinations of LLMs in public health,” *Front. Public Health*, vol. 13, Oct. 2025, doi: 10.3389/fpubh.2025.1635381.
- [13] H. Sansford, N. Richardson, H. P. Maretic, and J. N. Saada, “GraphEval: A Knowledge-Graph Based LLM Hallucination Evaluation Framework,” Jul. 2024, [Online]. Available: <http://arxiv.org/abs/2407.10793>
- [14] Y. Zheng, X. Liu, P. Wu, and L. Pan, “CRAVE: A Conflicting Reasoning Approach for Explainable Claim Verification Using LLMs,” in *2025 International Joint Conference on Neural Networks (IJCNN)*, IEEE, Jun. 2025, pp. 1–8. doi: 10.1109/IJCNN64981.2025.11227412.
- [15] B. Xu, F. Sun, X. Liu, P. Wu, X. Wang, and L. Pan, “Complex Claim Verification via Human Fact-Checking Imitation with Large Language Models,” in *2024 19th International Joint Symposium on Artificial Intelligence and Natural Language Processing (iSAI-NLP)*, IEEE, Nov. 2024, pp. 1–6. doi: 10.1109/iSAI-NLP64410.2024.10799276.
- [16] H.-Y. Huang, Y. Yang, Z. Zhang, S. Lee, and Y. Wu, “A Survey of Uncertainty Estimation in LLMs: Theory Meets Practice,” Oct. 2024, [Online]. Available: <http://arxiv.org/abs/2410.15326>
- [17] Z. Ding, X. Han, P. Liu, and M. Niethammer, “Local Temperature Scaling for Probability Calibration,” in *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, IEEE, Oct. 2021, pp. 6869–6879. doi: 10.1109/ICCV48922.2021.00681.
- [18] Z. Cao, Y. Li, and B.-S. Shin, “LogitAC: Logit Amplitude Constraints for Confidence Calibration and Out-of-Distribution Detection,” *IEEE Trans. Emerg. Top.*

- Comput. Intell.*, pp. 1–11, Aug. 2024, doi: 10.1109/TETCI.2024.3440196.
- [19] G. Hao, J. Wu, Q. Pan, and R. Morello, “Quantifying the uncertainty of LLM hallucination spreading in complex adaptive social networks,” *Sci. Rep.*, vol. 14, no. 1, Dec. 2024, doi: 10.1038/s41598-024-66708-4.
 - [20] W. H. Chin, Y. Guo, C. K. Loo, and N. Kubota, “A Hybrid Neuro-Symbolic Reasoning Framework for Automated Factual Verification in Conversational AI,” in *2025 International Conference on Machine Learning and Cybernetics (ICMLC)*, IEEE, Jul. 2025, pp. 315–320. doi: 10.1109/ICMLC66258.2025.11280070.
 - [21] F. Wang *et al.*, “A Comprehensive Survey of Small Language Models in the Era of Large Language Models: Techniques, Enhancements, Applications, Collaboration with LLMs, and Trustworthiness,” *ACM Trans. Intell. Syst. Technol.*, vol. 16, no. 6, Nov. 2025, doi: 10.1145/3768165.
 - [22] M. Hu *et al.*, “SLM Meets LLM: Balancing Latency, Interpretability and Consistency in Hallucination Detection,” Aug. 2024, [Online]. Available: <http://arxiv.org/abs/2408.12748>
 - [23] G. P. Reddy, Y. V. Pavan Kumar, and K. P. Prakash, “Hallucinations in Large Language Models (LLMs),” in *2024 IEEE Open Conference of Electrical, Electronic and Information Sciences (eStream)*, IEEE, Apr. 2024, pp. 1–6. doi: 10.1109/eStream61684.2024.10542617.
 - [24] X. Fang *et al.*, “Reframing Hallucination in Large Language Models: A Lifecycle-Based, Mechanism-Aligned, and Phenomenon-Consistent Definition,” in *2024 7th International Conference on Universal Village (UV)*, IEEE, Oct. 2024, pp. 1–15. doi: 10.1109/UV63228.2024.11189217.
 - [25] N. Delpisheh and Y. Chali, “Hallucinations in Abstractive Text Summarization with Large Language Models,” in *2025 2nd International Generative AI and Computational Language Modelling Conference (GACLM)*, IEEE, Aug. 2025, pp. 1–8. doi: 10.1109/GACLM67198.2025.11231924.
 - [26] J. Waldo and S. Boussard, “GPTs and Hallucination: Why do large language models hallucinate?,” *Queue*, vol. 22, no. 4, pp. 19–33, Aug. 2024, doi: 10.1145/3688007.
 - [27] Y. Zhang *et al.*, “The Law of Knowledge Overshadowing: Towards Understanding, Predicting, and Preventing LLM Hallucination,” Feb. 2025, [Online]. Available: <http://arxiv.org/abs/2502.16143>
 - [28] H. Orgad *et al.*, “LLMs Know More Than They Show: On the Intrinsic Representation of LLM Hallucinations,” May 2025, [Online]. Available: <http://arxiv.org/abs/2410.02707>

- [29] P. Tikka, J. Karjalainen, A. Alesani, and V. Goriachev, “Large Language Model hallucination mitigation in three industrial use cases,” *IEEE Access*, 2026, doi: 10.1109/ACCESS.2026.3659997.
- [30] S. Min *et al.*, “FactScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation,” Oct. 2023, [Online]. Available: <http://arxiv.org/abs/2305.14251>
- [31] Y. Bang *et al.*, “HalluLens: LLM Hallucination Benchmark,” Apr. 2025, [Online]. Available: <http://arxiv.org/abs/2504.17550>
- [32] A. Ravichander, S. Ghela, D. Wadden, and Y. Choi, “HALoGEN: Fantastic LLM Hallucinations and Where to Find Them,” Jan. 2025, [Online]. Available: <http://arxiv.org/abs/2501.08292>
- [33] X. Sun, J. Li, Y. Zhong, D. Zhao, and R. Yan, “Towards Detecting LLMs Hallucination via Markov Chain-based Multi-agent Debate Framework,” Jun. 2024, [Online]. Available: <http://arxiv.org/abs/2406.03075>
- [34] M. Cheung, “Hallucination Detection with Small Language Models,” Jun. 2025, [Online]. Available: <http://arxiv.org/abs/2506.22486>
- [35] B. N. Hiremath, S. Hiremath, N. R. N. J. N. V P, and G. S. Tejas, “RAGClin: A Privacy-Preserving Retrieval-Augmented Generation Framework for Clinical Question,” in *2025 5th International Conference on Emerging Research in Electronics, Computer Science and Technology (ICERECT)*, IEEE, Sep. 2025, pp. 1–6. doi: 10.1109/ICERECT65215.2025.11377112.
- [36] Y. Pu, Z. He, T. Qiu, H. Wu, and B. Yu, “Customized Retrieval Augmented Generation and Benchmarking for EDA Tool Documentation QA,” in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, New York, NY, USA: ACM, Oct. 2024, pp. 1–9. doi: 10.1145/3676536.3676730.
- [37] J. Niu, “ML-DPR: A Meta-Learning-Based Model for Domain-Adaptive Dense Passage Retrieval,” in *2023 IEEE International Conference on Image Processing and Computer Applications (ICIPCA)*, IEEE, Aug. 2023, pp. 985–989. doi: 10.1109/ICIPCA59209.2023.10257843.
- [38] K. Kim *et al.*, “Bauhaus: Restructuring Vector Database for LLM Retrieval on CXL-Based Tiered Memory,” *IEEE Transactions on Computers*, vol. 75, no. 4, pp. 1309–1322, Apr. 2026, doi: 10.1109/TC.2026.3656215.
- [39] W. Wu, H. Wang, B. Li, P. Huang, X. Zhao, and L. Liang, “MultiRAG: A Knowledge-guided Framework for Mitigating Hallucination in Multi-source Retrieval Augmented Generation,” Aug. 2025, doi: 10.1109/ICDE65448.2025.00230.

- [40] Y. Xu, T. Cai, J. Jiang, and X. Song, “Face4Rag: Factual Consistency Evaluation for Retrieval Augmented Generation in Chinese,” in *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, Association for Computing Machinery, Aug. 2024, pp. 6083–6094. doi: 10.1145/3637528.3671656.
- [41] K. Wolk, “Evaluating Retrieval-Augmented Generation Variants for Clinical Decision Support: Hallucination Mitigation and Secure On-Premises Deployment,” *Electronics (Switzerland)*, vol. 14, no. 21, Nov. 2025, doi: 10.3390/electronics14214227.
- [42] Y. Gato and T. Akiba, “A Retrieval-and-Classification Approach for Fact-checking Grounded by Assembly Minutes,” in *2022 9th International Conference on Advanced Informatics: Concepts, Theory and Applications (ICAICTA)*, IEEE, Sep. 2022, pp. 1–5. doi: 10.1109/ICAICTA56449.2022.9932924.
- [43] L. Zheng, C. Li, H. Jia, and X. Zhang, “Reasoning Paths as Signals: Augmenting Multi-hop Fact Verification through Structural Reasoning Progression,” *2025 IEEE 10th International Conference on Data Science in Cyberspace (DSC)*, pp. 359–366, Jun. 2025, [Online]. Available: <http://arxiv.org/abs/2506.07075>
- [44] A. Roy and A. Ekbal, “MulCoB-MulFaV: Multimodal Content Based Multilingual Fact Verification,” in *2021 International Joint Conference on Neural Networks (IJCNN)*, IEEE, Jul. 2021, pp. 1–8. doi: 10.1109/IJCNN52387.2021.9533916.
- [45] H. N. Dao, Y. Hashimoto, and T. C. Thang, “An LLM-Enabled Multi-Agent System for Evidence-Grounded Fact Checking,” in *2025 IEEE 18th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoc)*, IEEE, Dec. 2025, pp. 125–130. doi: 10.1109/MCSoc67473.2025.00029.
- [46] Z. Yan, J. Wang, J. Chen, X. Li, R. Li, and J. Z. Pan, “Atomic Fact Decomposition Helps Attributed Question Answering,” Sep. 2025, [Online]. Available: <http://arxiv.org/abs/2410.16708>
- [47] Z. Cao, Y. Yang, X. Li, and H. Zhao, “AutoHall: Automated Factuality Hallucination Dataset Generation for Large Language Models,” *IEEE Trans. Audio Speech Lang. Process.*, vol. 34, pp. 184–195, Jan. 2026, doi: 10.1109/TASL-PRO.2025.3635038.
- [48] E. Schönwälder, C. Falkenberg, C. Hartmann, and W. Lehner, “Abstention is all you need,” in *2025 IEEE 12th International Conference on Data Science and Advanced Analytics (DSAA)*, IEEE, Oct. 2025, pp. 1–10. doi: 10.1109/DSAA65442.2025.11248029.

- [49] Y. Huang *et al.*, “Look Before You Leap: An Exploratory Study of Uncertainty Analysis for Large Language Models,” *IEEE Transactions on Software Engineering*, vol. 51, no. 2, pp. 413–429, Jan. 2025, doi: 10.1109/TSE.2024.3519464.
- [50] C. Wang, Z. Cao, and K. Zhou, “Towards Faithful and Explainable Large Language Models: A Knowledge Graph-Guided Framework for Reasoning Enhancement and Hallucination Mitigation,” in *2025 IEEE/ACIS 29th International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)*, IEEE, Jun. 2025, pp. 92–99. doi: 10.1109/SNPD65828.2025.11253421.
- [51] R. Sullivan and N. Elsayed, “Can Large Language Models Act as Symbolic Reasoners?,” Oct. 2024, [Online]. Available: <http://arxiv.org/abs/2410.21490>
- [52] H. H. R. Rakshitha, R. M. S. R. C. Neeli, and R. S., “Structured Intelligence: Merging Neural and Symbolic AI,” in *2025 International Conference on Computing Technologies (ICOCT)*, IEEE, Jun. 2025, pp. 1–5. doi: 10.1109/ICOCT64433.2025.11118474.
- [53] Z. Xu, J. Li, and W. Zhang, “Large Language Model and Knowledge Graph Entangled Logical Reasoning,” in *2024 IEEE International Conference on Knowledge Graph (ICKG)*, IEEE, Dec. 2024, pp. 432–439. doi: 10.1109/ICKG63256.2024.00061.
- [54] G. Shrivastava, V. Veeraiah, S. Praveenkumar, P. Pathak, T. Sharma, and A. Gupta, “Explainable AI in NLP: Interpretable Models for Transparent Decision Making,” in *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, IEEE, Jun. 2024, pp. 1–6. doi: 10.1109/ICCCNT61001.2024.10726022.
- [55] E. Herron, J. Yin, and F. Wang, “SciTrust: Evaluating the Trustworthiness of Large Language Models for Science,” in *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, IEEE, Nov. 2024, pp. 72–78. doi: 10.1109/SCW63240.2024.00017.